

ZIP Code Geographic Analysis

Version 2.0

Generated by Doxygen 1.16.1

1 Class Index	1
1.1 Class List	1
2 File Index	2
2.1 File List	2
3 Class Documentation	3
3.1 FileHeader Struct Reference	3
3.1.1 Detailed Description	3
3.1.2 Member Data Documentation	4
3.2 HeaderBuffer Class Reference	6
3.2.1 Detailed Description	7
3.2.2 Constructor & Destructor Documentation	7
3.2.3 Member Function Documentation	7
3.2.4 Member Data Documentation	12
3.3 StateExtremes Struct Reference	12
3.3.1 Detailed Description	13
3.3.2 Constructor & Destructor Documentation	13
3.3.3 Member Data Documentation	13
3.4 ZipCodeBuffer Class Reference	15
3.4.1 Detailed Description	16
3.4.2 Constructor & Destructor Documentation	16
3.4.3 Member Function Documentation	17
3.4.4 Member Data Documentation	24
3.5 ZipCodeRecord Struct Reference	26
3.5.1 Detailed Description	27
3.5.2 Constructor & Destructor Documentation	27
3.5.3 Member Data Documentation	28
4 File Documentation	29
4.1 HeaderBuffer.cpp File Reference	29
4.1.1 Detailed Description	30
4.2 HeaderBuffer.cpp	30
4.3 HeaderBuffer.h File Reference	33
4.3.1 Detailed Description	33
4.4 HeaderBuffer.h	33
4.5 IndexBuilder.cpp File Reference	34
4.5.1 Detailed Description	35
4.5.2 Function Documentation	35
4.6 IndexBuilder.cpp	36

4.7 IndexBuilder.h File Reference	37
4.7.1 Detailed Description	37
4.7.2 Function Documentation	38
4.8 IndexBuilder.h	38
4.9 LenFileReader.cpp File Reference	39
4.9.1 Detailed Description	39
4.9.2 Function Documentation	39
4.10 LenFileReader.cpp	40
4.11 LenFileReader.h File Reference	41
4.11.1 Detailed Description	41
4.11.2 Function Documentation	42
4.12 LenFileReader.h	43
4.13 LenFileWriter.cpp File Reference	43
4.13.1 Detailed Description	44
4.13.2 Function Documentation	44
4.14 LenFileWriter.cpp	46
4.15 LenFileWriter.h File Reference	47
4.15.1 Detailed Description	47
4.15.2 Function Documentation	48
4.16 LenFileWriter.h	49
4.17 main.cpp File Reference	49
4.17.1 Detailed Description	50
4.17.2 Function Documentation	51
4.18 main.cpp	56
4.19 ZipCodeBuffer.cpp File Reference	64
4.19.1 Detailed Description	64
4.20 ZipCodeBuffer.cpp	64
4.21 ZipCodeBuffer.h File Reference	69
4.21.1 Detailed Description	69
4.22 ZipCodeBuffer.h	70

Index

73

1 Class Index

1.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

FileHeader	All header record fields required by the assignment	3
HeaderBuffer	Reads and writes the header record for a length-indicated file	6
StateExtremes	Keeps the most extreme ZIP codes for one state (east/west/north/south)	12
ZipCodeBuffer	A buffer class for reading ZIP code records from CSV files	15
ZipCodeRecord	Structure to hold a single ZIP code record	26

2 File Index

2.1 File List

Here is a list of all files with brief descriptions:

HeaderBuffer.cpp	Implementation of the HeaderBuffer class	29
HeaderBuffer.h	Defines the HeaderBuffer class and the FileHeader struct	33
IndexBuilder.cpp	Implementation for building ZIP->offset index files	34
IndexBuilder.h	Builds a primary key index file for a .len data file	37
LenFileReader.cpp	Implementation for reading length-indicated records	39
LenFileReader.h	Reads one record at a time from a length-indicated file (.len)	41
LenFileWriter.cpp	Implementation of CSV -> .len conversion	43
LenFileWriter.h	Converts a CSV file into a length-indicated file (.len)	47
main.cpp	Zip Code Group Project 2.0 main controller (CSV → LEN, Index, Search, Analyze)	49
ZipCodeBuffer.cpp	Implementation of the ZipCodeBuffer class	64
ZipCodeBuffer.h	Header file for the ZipCodeBuffer class and ZipCodeRecord structure	69

3 Class Documentation

3.1 FileHeader Struct Reference

All header record fields required by the assignment.

```
#include <HeaderBuffer.h>
```

Public Attributes

- bool [sizeIncludesItself](#)
true if size counts itself
- bool [staleIndex](#)
true if index may be out of date
- char [sizeFormatType](#)
'A' for ASCII, 'b' for binary
- int [version](#)
format version number, currently 2
- int [sizeOfSizes](#)
number of bytes used for record length
- int [headerSizeBytes](#)
size of header record in bytes
- int [recordSizeByteCount](#)
bytes used for each record size integer
- int [fieldCount](#)
number of fields in each record
- int [primaryKeyFieldIndex](#)
0-based index of the primary key field
- long [recordCount](#)
total number of data records
- string [fileType](#)
name of file type, e.g. "ZipLenFile"
- string [indexFileName](#)
name of the .idx file
- vector< string > [fieldNames](#)
the names of every field, e.g. "PlaceName"
- vector< string > [fieldTypes](#)
the data types of every field, e.g. "int"

3.1.1 Detailed Description

All header record fields required by the assignment.

Definition at line 22 of file [HeaderBuffer.h](#).

3.1.2 Member Data Documentation

fieldCount

```
int FileHeader::fieldCount
```

number of fields in each record

Definition at line 30 of file [HeaderBuffer.h](#).

fieldNames

```
vector<string> FileHeader::fieldNames
```

the names of every field, e.g. "PlaceName"

Definition at line 35 of file [HeaderBuffer.h](#).

fieldTypes

```
vector<string> FileHeader::fieldTypes
```

the data types of every field, e.g. "int"

Definition at line 36 of file [HeaderBuffer.h](#).

fileType

```
string FileHeader::fileType
```

name of file type, e.g. "ZipLenFile"

Definition at line 33 of file [HeaderBuffer.h](#).

headerSizeBytes

```
int FileHeader::headerSizeBytes
```

size of header record in bytes

Definition at line 28 of file [HeaderBuffer.h](#).

indexFileName

```
string FileHeader::indexFileName
```

name of the .idx file

Definition at line 34 of file [HeaderBuffer.h](#).

primaryKeyFieldIndex

```
int FileHeader::primaryKeyFieldIndex
```

0-based index of the primary key field

Definition at line 31 of file [HeaderBuffer.h](#).

recordCount

```
long FileHeader::recordCount
```

total number of data records

Definition at line 32 of file [HeaderBuffer.h](#).

recordSizeByteCount

```
int FileHeader::recordSizeByteCount
```

bytes used for each record size integer

Definition at line 29 of file [HeaderBuffer.h](#).

sizeFormatType

```
char FileHeader::sizeFormatType
```

'A' for ASCII, 'b' for binary

Definition at line 25 of file [HeaderBuffer.h](#).

sizeIncludesItself

```
bool FileHeader::sizeIncludesItself
```

true if size counts itself

Definition at line 23 of file [HeaderBuffer.h](#).

sizeofSizes

```
int FileHeader::sizeofSizes
```

number of bytes used for record length

Definition at line 27 of file [HeaderBuffer.h](#).

staleIndex

```
bool FileHeader::staleIndex
```

true if index may be out of date

Definition at line 24 of file [HeaderBuffer.h](#).

version

```
int FileHeader::version
```

format version number, currently 2

Definition at line 26 of file [HeaderBuffer.h](#).

The documentation for this struct was generated from the following file:

- [HeaderBuffer.h](#)

3.2 HeaderBuffer Class Reference

Reads and writes the header record for a length-indicated file.

```
#include <HeaderBuffer.h>
```

Public Member Functions

- [HeaderBuffer](#) ()
Default constructor for the [HeaderBuffer](#) class.
- void [buildDefault](#) (const string &indexFileName, long recordCount)
Build a default header for the ZIP code file.
- bool [write](#) (ofstream &out)
Write header to an open output stream.
- bool [read](#) (ifstream &in)
Read header from an open input stream.
- const [FileHeader](#) & [getHeader](#) () const
Get the loaded header data.
- void [print](#) () const
Print header contents to cout.

Private Member Functions

- string [serialize](#) () const
Serializes the header structure into a comma separated string.
- bool [deserialize](#) (const string &s)
Converts a serialized header string into a header structure.
- bool [writeLenLine](#) (ofstream &out, const string &text)
Writes a length indicated record to the output file.
- bool [readLenLine](#) (ifstream &in, string &text)
Reads a length indicated record from the input file.

Private Attributes

- [FileHeader](#) [header_](#)

3.2.1 Detailed Description

Reads and writes the header record for a length-indicated file.

Definition at line 43 of file [HeaderBuffer.h](#).

3.2.2 Constructor & Destructor Documentation

HeaderBuffer()

```
HeaderBuffer::HeaderBuffer ()
```

Default constructor for the [HeaderBuffer](#) class.

Definition at line 21 of file [HeaderBuffer.cpp](#).

3.2.3 Member Function Documentation

buildDefault()

```
void HeaderBuffer::buildDefault (  
    const string & indexFileName,  
    long recordCount)
```

Build a default header for the ZIP code file.

Builds a default header structure.

Initializes the file header metadata including file type, version number, record size formatting, index file name, record count, and field descriptions for Zip Code records.

Parameters

<i>indexFileName</i>	Name of the primary key index file.
----------------------	-------------------------------------

<i>recordCount</i>	Number of records stored in the data file.
--------------------	--

Definition at line 34 of file [HeaderBuffer.cpp](#).

References [header_](#), and [serialize\(\)](#).

Referenced by [makeLenFromCsv\(\)](#).

deserialize()

```
bool HeaderBuffer::deserialize (  
    const string & s) [private]
```

Converts a serialized header string into a header structure.

Parses the comma separated metadata and reconstructs the [FileHeader](#) structure including field descriptors.

Parameters

<i>s</i>	Serialized header string.
----------	---------------------------

Returns

True if the header was successfully parsed.

Definition at line 144 of file [HeaderBuffer.cpp](#).

References [header_](#).

Referenced by [read\(\)](#).

getHeader()

```
const FileHeader & HeaderBuffer::getHeader () const
```

Get the loaded header data.

Retrieves the current file header.

Returns

Constant reference to the [FileHeader](#) structure.

Definition at line 56 of file [HeaderBuffer.cpp](#).

References [header_](#).

print()

```
void HeaderBuffer::print () const
```

Print header contents to cout.

Prints the header metadata to the console.

This function is for debugging and verifying the contents of the file header.

Definition at line 90 of file [HeaderBuffer.cpp](#).

References [header_](#).

read()

```
bool HeaderBuffer::read (  
    ifstream & in)
```

Read header from an open input stream.

Reads the header record from an input file.

Parameters

<i>in</i>	Input file stream.
-----------	--------------------

Returns

True if the header was successfully read and parsed.

Definition at line 78 of file [HeaderBuffer.cpp](#).

References [deserialize\(\)](#), and [readLenLine\(\)](#).

readLenLine()

```
bool HeaderBuffer::readLenLine (  
    ifstream & in,  
    string & text) [private]
```

Reads a length indicated record from the input file.

The function first reads a fixed-width numeric field indicating the size of the following record, then reads the record text.

Parameters

<i>in</i>	Input file stream.
-----------	--------------------

<i>text</i>	Output variable storing the record text.
-------------	--

Returns

True if the record was successfully read.

Definition at line 194 of file [HeaderBuffer.cpp](#).

Referenced by [read\(\)](#).

serialize()

```
string HeaderBuffer::serialize () const [private]
```

Serializes the header structure into a comma separated string.

The serialized string begins with the identifier "HDR" followed by all metadata values and field definitions.

Returns

Serialized header string.

Definition at line 115 of file [HeaderBuffer.cpp](#).

References [header_](#).

Referenced by [buildDefault\(\)](#), and [write\(\)](#).

write()

```
bool HeaderBuffer::write (  
    ofstream & out)
```

Write header to an open output stream.

Writes the header record to the output file.

The header is serialized and written using a length indicated record format.

Parameters

<i>out</i>	Output file stream.
------------	---------------------

Returns

True if the header was written successfully.

Definition at line 68 of file [HeaderBuffer.cpp](#).

References [serialize\(\)](#), and [writeLenLine\(\)](#).

Referenced by [makeLenFromCsv\(\)](#).

writeLenLine()

```
bool HeaderBuffer::writeLenLine (
    ostream & out,
    const string & text) [private]
```

Writes a length indicated record to the output file.

Parameters

<i>out</i>	Output file stream.
------------	---------------------

<i>text</i>	Text content to write.
-------------	------------------------

Returns

True if the operation succeeded.

Definition at line 179 of file [HeaderBuffer.cpp](#).

Referenced by [write\(\)](#).

3.2.4 Member Data Documentation

header_

[FileHeader](#) `HeaderBuffer::header_` [private]

Definition at line 63 of file [HeaderBuffer.h](#).

Referenced by [buildDefault\(\)](#), [deserialize\(\)](#), [getHeader\(\)](#), [print\(\)](#), and [serialize\(\)](#).

The documentation for this class was generated from the following files:

- [HeaderBuffer.h](#)
- [HeaderBuffer.cpp](#)

3.3 StateExtremes Struct Reference

Keeps the most extreme ZIP codes for one state (east/west/north/south).

Public Member Functions

- [StateExtremes](#) ()

Public Attributes

- int [easternmost](#)
- int [westernmost](#)
- int [northernmost](#)
- int [southernmost](#)
- double [minLongitude](#)
- double [maxLongitude](#)
- double [maxLatitude](#)
- double [minLatitude](#)

3.3.1 Detailed Description

Keeps the most extreme ZIP codes for one state (east/west/north/south).

We store:

- Which ZIP code is easternmost / westernmost / northernmost / southernmost
- The coordinate values used to compare

Tie-breaking rule:

- If two ZIPs tie on coordinate, choose the smaller ZIP so results are stable even if the input rows are shuffled.

Definition at line 68 of file [main.cpp](#).

3.3.2 Constructor & Destructor Documentation

StateExtremes()

```
StateExtremes::StateExtremes () [inline]
```

Definition at line 79 of file [main.cpp](#).

References [easternmost](#), [maxLatitude](#), [maxLongitude](#), [minLatitude](#), [minLongitude](#), [northernmost](#), [southernmost](#), and [westernmost](#).

3.3.3 Member Data Documentation

easternmost

```
int StateExtremes::easternmost
```

Definition at line 69 of file [main.cpp](#).

Referenced by [printStateExtremesTable\(\)](#), [StateExtremes\(\)](#), and [updateStateExtremes\(\)](#).

maxLatitude

```
double StateExtremes::maxLatitude
```

Definition at line 76 of file [main.cpp](#).

Referenced by [StateExtremes\(\)](#), and [updateStateExtremes\(\)](#).

maxLongitude

```
double StateExtremes::maxLongitude
```

Definition at line 75 of file [main.cpp](#).

Referenced by [StateExtremes\(\)](#), and [updateStateExtremes\(\)](#).

minLatitude

```
double StateExtremes::minLatitude
```

Definition at line 77 of file [main.cpp](#).

Referenced by [StateExtremes\(\)](#), and [updateStateExtremes\(\)](#).

minLongitude

```
double StateExtremes::minLongitude
```

Definition at line 74 of file [main.cpp](#).

Referenced by [StateExtremes\(\)](#), and [updateStateExtremes\(\)](#).

northernmost

```
int StateExtremes::northernmost
```

Definition at line 71 of file [main.cpp](#).

Referenced by [printStateExtremesTable\(\)](#), [StateExtremes\(\)](#), and [updateStateExtremes\(\)](#).

southernmost

```
int StateExtremes::southernmost
```

Definition at line 72 of file [main.cpp](#).

Referenced by [printStateExtremesTable\(\)](#), [StateExtremes\(\)](#), and [updateStateExtremes\(\)](#).

westernmost

```
int StateExtremes::westernmost
```

Definition at line 70 of file [main.cpp](#).

Referenced by [printStatsExtremesTable\(\)](#), [StateExtremes\(\)](#), and [updateStateExtremes\(\)](#).

The documentation for this struct was generated from the following file:

- [main.cpp](#)

3.4 ZipCodeBuffer Class Reference

A buffer class for reading ZIP code records from CSV files.

```
#include <ZipCodeBuffer.h>
```

Public Member Functions

- [ZipCodeBuffer](#) ()
Default constructor.
- [ZipCodeBuffer](#) (const string &csvFilename)
Constructor that immediately opens a CSV file.
- [~ZipCodeBuffer](#) ()
Destructor.
- bool [open](#) (const string &csvFilename)
Opens a CSV file for reading.
- void [close](#) ()
Closes the file and resets internal state.
- bool [isOpen](#) () const
Checks whether a file is currently open.
- bool [readRecord](#) ([ZipCodeRecord](#) &record)
Reads the next ZIP code record from the file.
- vector< [ZipCodeRecord](#) > [gatherAllRecords](#) ()
Reads all records into a vector.
- bool [reset](#) ()
Resets the file position to the beginning, then processes header.
- long [getRecordCount](#) () const
Gets the number of records read so far.
- string [getFilename](#) () const
Gets the current filename.

Private Member Functions

- bool [parseLine](#) (const string &line, [ZipCodeRecord](#) &record)
Parses a CSV line into a [ZipCodeRecord](#).
- vector< string > [splitCSV](#) (const string &line)
Splits a CSV line into separate fields.
- string [trim](#) (const string &str)
Removes whitespace from both ends of a string.
- bool [parseHeader](#) (const string &headerLine)
Reads the header row and finds the column position of each field.

Private Attributes

- ifstream [fileStream](#)
Input file stream for CSV data.
- string [filename](#)
Name of the current file.
- bool [headerSkipped](#)
True after header row is processed.
- long [recordCount](#)
Number of records successfully read.
- int [colZip](#)
- int [colPlace](#)
- int [colState](#)
- int [colCounty](#)
- int [colLat](#)
- int [colLong](#)

3.4.1 Detailed Description

A buffer class for reading ZIP code records from CSV files.

This class opens a CSV file, reads one record at a time, and converts each line into a [ZipCodeRecord](#) object.

Project 2 improvement: The class does not assume that columns are always in the same order. Instead, it reads the header row and finds where each needed field is.

Definition at line [79](#) of file [ZipCodeBuffer.h](#).

3.4.2 Constructor & Destructor Documentation

[ZipCodeBuffer\(\)](#) [1/2]

```
ZipCodeBuffer::ZipCodeBuffer ()
```

Default constructor.

Initializes all member variables to safe starting values.

Definition at line [49](#) of file [ZipCodeBuffer.cpp](#).

References [colCounty](#), [colLat](#), [colLong](#), [colPlace](#), [colState](#), [colZip](#), [filename](#), [headerSkipped](#), and [recordCount](#).

ZipCodeBuffer() [2/2]

```
ZipCodeBuffer::ZipCodeBuffer (
    const string & csvFilename) [explicit]
```

Constructor that immediately opens a CSV file.

Constructor that opens a file immediately.

Parameters

<i>csvFilename</i>	Path to CSV file
--------------------	------------------

Definition at line 58 of file [ZipCodeBuffer.cpp](#).

References [colCounty](#), [colLat](#), [colLong](#), [colPlace](#), [colState](#), [colZip](#), [filename](#), [headerSkipped](#), [open\(\)](#), and [recordCount](#).

~ZipCodeBuffer()

```
ZipCodeBuffer::~ZipCodeBuffer ()
```

Destructor.

Definition at line 68 of file [ZipCodeBuffer.cpp](#).

References [close\(\)](#).

3.4.3 Member Function Documentation**close()**

```
void ZipCodeBuffer::close ()
```

Closes the file and resets internal state.

Definition at line 113 of file [ZipCodeBuffer.cpp](#).

References [colCounty](#), [colLat](#), [colLong](#), [colPlace](#), [colState](#), [colZip](#), [filename](#), [fileStream](#), [headerSkipped](#), and [recordCount](#).

Referenced by [analyzeCsvStreaming\(\)](#), [open\(\)](#), and [~ZipCodeBuffer\(\)](#).

gatherAllRecords()

```
vector< ZipCodeRecord > ZipCodeBuffer::gatherAllRecords ()
```

Reads all records into a vector.

Returns

Vector of all records

Note: This is kept mainly for compatibility with older code, but Project 2 should prefer reading one record at a time.

This is kept for compatibility, but Project 2 should prefer reading one record at a time.

Definition at line [171](#) of file [ZipCodeBuffer.cpp](#).

References [readRecord\(\)](#), and [reset\(\)](#).

getFilename()

```
string ZipCodeBuffer::getFilename () const
```

Gets the current filename.

Returns current filename.

Returns

Filename

Definition at line [222](#) of file [ZipCodeBuffer.cpp](#).

References [filename](#).

getRecordCount()

```
long ZipCodeBuffer::getRecordCount () const
```

Gets the number of records read so far.

Returns how many records have been read.

Returns

Record count

Definition at line [215](#) of file [ZipCodeBuffer.cpp](#).

References [recordCount](#).

isOpen()

```
bool ZipCodeBuffer::isOpen () const
```

Checks whether a file is currently open.

Checks whether file is open.

Returns

true if file is open

Definition at line 133 of file [ZipCodeBuffer.cpp](#).

References [fileStream](#).

open()

```
bool ZipCodeBuffer::open (  
    const string & csvFilename)
```

Opens a CSV file for reading.

Opens a CSV file and processes its header row.

Parameters

<i>csvFilename</i>	Path to CSV file
--------------------	------------------

Returns

true if file opened successfully, false otherwise

This function: 1) closes any previously open file 2) opens the requested file 3) reads the first line (header row) 4) finds the positions of ZIP, place, state, county, latitude, longitude

Parameters

<i>csvFilename</i>	CSV file path
--------------------	---------------

Returns

true on success, false on failure

Definition at line 84 of file [ZipCodeBuffer.cpp](#).

References [close\(\)](#), [filename](#), [fileStream](#), [headerSkipped](#), [parseHeader\(\)](#), and [recordCount](#).

Referenced by [analyzeCsvStreaming\(\)](#), and [ZipCodeBuffer\(\)](#).

parseHeader()

```
bool ZipCodeBuffer::parseHeader (
    const string & headerLine) [private]
```

Reads the header row and finds the column position of each field.

Parses the CSV header row and finds column positions.

Parameters

<i>headerLine</i>	The first line of the CSV file
-------------------	--------------------------------

Returns

true if all required columns were found, false otherwise

This function makes the class work even when the CSV columns are re-ordered.

This function makes column re-ordering possible.

For example, if the header is: PlaceName,ZipCode,County,State,Long,Lat

then this function will correctly discover:

- colPlace = 0
- colZip = 1
- colCounty = 2
- colState = 3
- colLong = 4
- colLat = 5

Parameters

<i>headerLine</i>	Header row from the CSV file
-------------------	------------------------------

Returns

true if all needed columns are found

Definition at line [245](#) of file [ZipCodeBuffer.cpp](#).

References [colCounty](#), [colLat](#), [colLong](#), [colPlace](#), [colState](#), [colZip](#), [splitCSV\(\)](#), and [trim\(\)](#).

Referenced by [open\(\)](#), and [reset\(\)](#).

parseLine()

```
bool ZipCodeBuffer::parseLine (  
    const string & line,  
    ZipCodeRecord & record) [private]
```

Parses a CSV line into a [ZipCodeRecord](#).

Parses one CSV data line into a [ZipCodeRecord](#).

Parameters

<i>line</i>	The CSV line to parse
-------------	-----------------------

<i>record</i>	Reference to record to fill
---------------	-----------------------------

Returns

true if parsing succeeds, false otherwise

This version does NOT assume fixed column order. Instead, it uses the column positions discovered from the header row.

Parameters

<i>line</i>	One CSV data row
<i>record</i>	Output record

Returns

true if parsing succeeds

Definition at line 296 of file [ZipCodeBuffer.cpp](#).

References [colCounty](#), [colLat](#), [colLong](#), [colPlace](#), [colState](#), [colZip](#), [ZipCodeRecord::county](#), [ZipCodeRecord::latitude](#), [ZipCodeRecord::longitude](#), [ZipCodeRecord::placeName](#), [splitCSV\(\)](#), [ZipCodeRecord::state](#), [trim\(\)](#), and [ZipCodeRecord::zipCode](#).

Referenced by [readRecord\(\)](#).

readRecord()

```
bool ZipCodeBuffer::readRecord (  
    ZipCodeRecord & record)
```

Reads the next ZIP code record from the file.

Reads one data record from the CSV file.

Parameters

<i>record</i>	Output record
---------------	---------------

Returns

true if record was read successfully, false on EOF or error

Parameters

<i>record</i>	Output record
---------------	---------------

Returns

true if record is read successfully, false on EOF or parse failure

Definition at line 143 of file [ZipCodeBuffer.cpp](#).

References [fileStream](#), [parseLine\(\)](#), [readRecord\(\)](#), [recordCount](#), and [trim\(\)](#).

Referenced by [analyzeCsvStreaming\(\)](#), [gatherAllRecords\(\)](#), and [readRecord\(\)](#).

reset()

```
bool ZipCodeBuffer::reset ()
```

Resets the file position to the beginning, then processes header.

Resets file back to beginning and processes header row again.

Returns

true if reset succeeds, false otherwise

true if reset succeeds

Definition at line 190 of file [ZipCodeBuffer.cpp](#).

References [fileStream](#), [headerSkipped](#), [parseHeader\(\)](#), and [recordCount](#).

Referenced by [gatherAllRecords\(\)](#).

splitCSV()

```
vector< string > ZipCodeBuffer::splitCSV (  
    const string & line) [private]
```

Splits a CSV line into separate fields.

Splits a CSV line into fields.

Parameters

<i>line</i>	One CSV line
-------------	--------------

Returns

Vector of field strings

Handles:

- normal commas
- quoted text
- commas inside quoted text

Parameters

<i>line</i>	One CSV line
-------------	--------------

Returns

Vector of fields

Definition at line 329 of file [ZipCodeBuffer.cpp](#).

Referenced by [parseHeader\(\)](#), and [parseLine\(\)](#).

trim()

```
string ZipCodeBuffer::trim (  
    const string & str) [private]
```

Removes whitespace from both ends of a string.

Trims whitespace from both ends of a string.

Parameters

<i>str</i>	Input string
------------	--------------

Returns

Trimmed string

Definition at line [357](#) of file [ZipCodeBuffer.cpp](#).

Referenced by [parseHeader\(\)](#), [parseLine\(\)](#), and [readRecord\(\)](#).

3.4.4 Member Data Documentation

colCounty

```
int ZipCodeBuffer::colCounty [private]
```

Definition at line [100](#) of file [ZipCodeBuffer.h](#).

Referenced by [close\(\)](#), [parseHeader\(\)](#), [parseLine\(\)](#), [ZipCodeBuffer\(\)](#), and [ZipCodeBuffer\(\)](#).

colLat

```
int ZipCodeBuffer::colLat [private]
```

Definition at line [101](#) of file [ZipCodeBuffer.h](#).

Referenced by [close\(\)](#), [parseHeader\(\)](#), [parseLine\(\)](#), [ZipCodeBuffer\(\)](#), and [ZipCodeBuffer\(\)](#).

colLong

```
int ZipCodeBuffer::colLong [private]
```

Definition at line [102](#) of file [ZipCodeBuffer.h](#).

Referenced by [close\(\)](#), [parseHeader\(\)](#), [parseLine\(\)](#), [ZipCodeBuffer\(\)](#), and [ZipCodeBuffer\(\)](#).

colPlace

```
int ZipCodeBuffer::colPlace [private]
```

Definition at line 98 of file [ZipCodeBuffer.h](#).

Referenced by [close\(\)](#), [parseHeader\(\)](#), [parseLine\(\)](#), [ZipCodeBuffer\(\)](#), and [ZipCodeBuffer\(\)](#).

colState

```
int ZipCodeBuffer::colState [private]
```

Definition at line 99 of file [ZipCodeBuffer.h](#).

Referenced by [close\(\)](#), [parseHeader\(\)](#), [parseLine\(\)](#), [ZipCodeBuffer\(\)](#), and [ZipCodeBuffer\(\)](#).

colZip

```
int ZipCodeBuffer::colZip [private]
```

Column positions found from the CSV header row.

These values tell the parser where each needed field is located. Example:

- colZip = 0 means ZIP code is in column 0
- colState = 2 means state is in column 2

If the columns are re-ordered, these values will change, and parsing will still work correctly.

Definition at line 97 of file [ZipCodeBuffer.h](#).

Referenced by [close\(\)](#), [parseHeader\(\)](#), [parseLine\(\)](#), [ZipCodeBuffer\(\)](#), and [ZipCodeBuffer\(\)](#).

filename

```
string ZipCodeBuffer::filename [private]
```

Name of the current file.

Definition at line 82 of file [ZipCodeBuffer.h](#).

Referenced by [close\(\)](#), [getFilename\(\)](#), [open\(\)](#), [ZipCodeBuffer\(\)](#), and [ZipCodeBuffer\(\)](#).

fileStream

```
ifstream ZipCodeBuffer::fileStream [private]
```

Input file stream for CSV data.

Definition at line 81 of file [ZipCodeBuffer.h](#).

Referenced by [close\(\)](#), [isOpen\(\)](#), [open\(\)](#), [readRecord\(\)](#), and [reset\(\)](#).

headerSkipped

```
bool ZipCodeBuffer::headerSkipped [private]
```

True after header row is processed.

Definition at line 83 of file [ZipCodeBuffer.h](#).

Referenced by [close\(\)](#), [open\(\)](#), [reset\(\)](#), [ZipCodeBuffer\(\)](#), and [ZipCodeBuffer\(\)](#).

recordCount

```
long ZipCodeBuffer::recordCount [private]
```

Number of records successfully read.

Definition at line 84 of file [ZipCodeBuffer.h](#).

Referenced by [close\(\)](#), [getRecordCount\(\)](#), [open\(\)](#), [readRecord\(\)](#), [reset\(\)](#), [ZipCodeBuffer\(\)](#), and [ZipCodeBuffer\(\)](#).

The documentation for this class was generated from the following files:

- [ZipCodeBuffer.h](#)
- [ZipCodeBuffer.cpp](#)

3.5 ZipCodeRecord Struct Reference

Structure to hold a single ZIP code record.

```
#include <ZipCodeBuffer.h>
```

Public Member Functions

- [ZipCodeRecord](#) ()
Default constructor.
- [ZipCodeRecord](#) (int zip, const string &place, const string &st, const string &cnty, double lat, double lon)
Parameterized constructor.

Public Attributes

- int [zipCode](#)
The 5-digit ZIP code.
- string [placeName](#)
Name of the place/city.
- string [state](#)
Two-letter state abbreviation.
- string [county](#)
County name.
- double [latitude](#)
Latitude coordinate.
- double [longitude](#)
Longitude coordinate.

3.5.1 Detailed Description

Structure to hold a single ZIP code record.

This structure stores one full data row from the ZIP code file.

Definition at line 40 of file [ZipCodeBuffer.h](#).

3.5.2 Constructor & Destructor Documentation

ZipCodeRecord() [1/2]

```
ZipCodeRecord::ZipCodeRecord ()
```

Default constructor.

Initializes all values to safe defaults.

Definition at line 25 of file [ZipCodeBuffer.cpp](#).

References [county](#), [latitude](#), [longitude](#), [placeName](#), [state](#), and [zipCode](#).

ZipCodeRecord() [2/2]

```
ZipCodeRecord::ZipCodeRecord (  
    int zip,  
    const string & place,  
    const string & st,  
    const string & cnty,  
    double lat,  
    double lon)
```

Parameterized constructor.

Parameters

<i>zip</i>	ZIP code number
------------	-----------------

<i>place</i>	Place name
<i>st</i>	State abbreviation
<i>cnty</i>	County name
<i>lat</i>	Latitude
<i>lon</i>	Longitude

Definition at line 33 of file [ZipCodeBuffer.cpp](#).

References [county](#), [latitude](#), [longitude](#), [placeName](#), [state](#), and [zipCode](#).

3.5.3 Member Data Documentation

county

```
string ZipCodeRecord::county
```

County name.

Definition at line 44 of file [ZipCodeBuffer.h](#).

Referenced by [ZipCodeBuffer::parseLine\(\)](#), [ZipCodeRecord\(\)](#), and [ZipCodeRecord\(\)](#).

latitude

```
double ZipCodeRecord::latitude
```

Latitude coordinate.

Definition at line 45 of file [ZipCodeBuffer.h](#).

Referenced by [ZipCodeBuffer::parseLine\(\)](#), [updateStateExtremes\(\)](#), [ZipCodeRecord\(\)](#), and [ZipCodeRecord\(\)](#).

longitude

```
double ZipCodeRecord::longitude
```

Longitude coordinate.

Definition at line 46 of file [ZipCodeBuffer.h](#).

Referenced by [ZipCodeBuffer::parseLine\(\)](#), [updateStateExtremes\(\)](#), [ZipCodeRecord\(\)](#), and [ZipCodeRecord\(\)](#).

placeName

```
string ZipCodeRecord::placeName
```

Name of the place/city.

Definition at line 42 of file [ZipCodeBuffer.h](#).

Referenced by [ZipCodeBuffer::parseLine\(\)](#), [ZipCodeRecord\(\)](#), and [ZipCodeRecord\(\)](#).

state

```
string ZipCodeRecord::state
```

Two-letter state abbreviation.

Definition at line 43 of file [ZipCodeBuffer.h](#).

Referenced by [ZipCodeBuffer::parseLine\(\)](#), [updateStateExtremes\(\)](#), [ZipCodeRecord\(\)](#), and [ZipCodeRecord\(\)](#).

zipCode

```
int ZipCodeRecord::zipCode
```

The 5-digit ZIP code.

Definition at line 41 of file [ZipCodeBuffer.h](#).

Referenced by [ZipCodeBuffer::parseLine\(\)](#), [updateStateExtremes\(\)](#), [ZipCodeRecord\(\)](#), and [ZipCodeRecord\(\)](#).

The documentation for this struct was generated from the following files:

- [ZipCodeBuffer.h](#)
- [ZipCodeBuffer.cpp](#)

4 File Documentation

4.1 HeaderBuffer.cpp File Reference

implementation of the [HeaderBuffer](#) class

```
#include "HeaderBuffer.h"
#include <sstream>
#include <iostream>
#include <iomanip>
#include <cctype>
```

4.1.1 Detailed Description

implementation of the [HeaderBuffer](#) class

Author

Dristi Barnwal (original author)

Ethan Jackson (refactoring and additional comments)

Marcus Julius, Teagen Lee, Natoli Mayu (reviewers)

Date

March 2026

Definition in file [HeaderBuffer.cpp](#).

4.2 HeaderBuffer.cpp

[Go to the documentation of this file.](#)

```
00001 /**
00002  * @file HeaderBuffer.cpp
00003  * @brief implementation of the HeaderBuffer class
00004  * @author Dristi Barnwal (original author)
00005  * @author Ethan Jackson (refactoring and additional comments)
00006  * @author Marcus Julius, Teagen Lee, Natoli Mayu (reviewers)
00007  * @date March 2026
00008  */
00009 #include "HeaderBuffer.h"
00010 #include <sstream>
00011 #include <iostream>
00012 #include <iomanip>
00013 #include <cctype>
00014
00015 using namespace std;
00016
00017 /**
00018  * @brief Default constructor for the HeaderBuffer class.
00019  */
00020
00021 HeaderBuffer::HeaderBuffer() {}
00022
00023 /**
00024  * @brief Builds a default header structure.
00025  *
00026  * Initializes the file header metadata including file type,
00027  * version number, record size formatting, index file name,
00028  * record count, and field descriptions for Zip Code records.
00029  *
00030  * @param indexFileName Name of the primary key index file.
00031  * @param recordCount Number of records stored in the data file.
00032  */
00033
00034 void HeaderBuffer::buildDefault(const string& indexFileName, long recordCount) {
00035     header_.fileType = "ZipLenFile";
00036     header_.version = 2;
00037     header_.recordSizeByteCount = 22; //minimum record size (State is 2 chars)
00038     header_.sizeFormatType = 'A';
00039     header_.sizeOfSizes = 10; //10 digits in ASCII
00040     header_.sizeIncludesItself = true; //based on how header size is calculated
00041     header_.indexFileName = indexFileName;
00042     header_.recordCount = recordCount;
00043     header_.primaryKeyFieldIndex = 0;
00044     header_.staleIndex = false;
00045     header_.fieldNames = {"ZipCode", "PlaceName", "State", "County", //continued
```



```

00046         "Longitude", "Latitude"}};
00047     header_.fieldTypes = {"int","string","string","string","double","double"};
00048     header_.fieldCount = (int)header_.fieldNames.size();
00049     header_.headerSizeBytes = (int)serialize().size();
00050 }
00051 /**
00052  * @brief Retrieves the current file header.
00053  *
00054  * @return Constant reference to the FileHeader structure.
00055  */
00056 const FileHeader& HeaderBuffer::getHeader() const {
00057     return header_;
00058 }
00059 /**
00060  * @brief Writes the header record to the output file.
00061  *
00062  * The header is serialized and written using a length indicated
00063  * record format.
00064  *
00065  * @param out Output file stream.
00066  * @return True if the header was written successfully.
00067  */
00068 bool HeaderBuffer::write(ofstream& out) {
00069     return writeLenLine(out, serialize());
00070 }
00071
00072 /**
00073  * @brief Reads the header record from an input file.
00074  *
00075  * @param in Input file stream.
00076  * @return True if the header was successfully read and parsed.
00077  */
00078 bool HeaderBuffer::read(ifstream& in) {
00079     string s;
00080     if (!readLenLine(in, s)) return false;
00081     return deserialize(s);
00082 }
00083
00084 /**
00085  * @brief Prints the header metadata to the console.
00086  *
00087  * This function is for debugging and verifying
00088  * the contents of the file header.
00089  */
00090 void HeaderBuffer::print() const {
00091     cout << "File type:\t    " << header_.fileType << "\n";
00092     cout << "Version:\t    " << header_.version << "\n";
00093     cout << "Header size:\t    " << header_.headerSizeBytes << " bytes\n";
00094     cout << "Size format:\t    " << header_.sizeFormatType << "\n";
00095     cout << "Size of sizes:\t    " << header_.sizeOfSizes << " bytes\n";
00096     cout << "Size counts itself: " << (header_.sizeIncludesItself?"yes":"no") << "\n";
00097     cout << "Index file:\t    " << header_.indexFileName << "\n";
00098     cout << "Record count:\t    " << header_.recordCount << "\n";
00099     cout << "Field count:\t    " << header_.fieldCount << "\n";
00100     cout << "Primary key field: " << header_.primaryKeyFieldIndex << "\n";
00101     cout << "Stale index:\t    " << (header_.staleIndex ? "yes" : "no") << "\n";
00102     for (int i = 0; i < (int)header_.fieldCount; i++) {
00103         cout << "Field[" << i << "]:\t    " << header_.fieldNames[i]
00104             << " (" << header_.fieldTypes[i] << ")\n";
00105     }
00106 }
00107 /**
00108  * @brief Serializes the header structure into a comma separated string.
00109  *
00110  * The serialized string begins with the identifier "HDR" followed
00111  * by all metadata values and field definitions.
00112  *
00113  * @return Serialized header string.
00114  */
00115 string HeaderBuffer::serialize() const {
00116     ostringstream ss;
00117     ss << "HDR"
00118         << "," << header_.fileType
00119         << "," << header_.version
00120         << "," << header_.recordSizeByteCount
00121         << "," << header_.sizeFormatType
00122         << "," << header_.sizeOfSizes
00123         << "," << (header_.sizeIncludesItself ? "1" : "0")
00124         << "," << header_.indexFileName
00125         << "," << header_.recordCount
00126         << "," << header_.fieldCount

```

```

00127     « "," « header_.primaryKeyFieldIndex
00128     « "," « (header_.staleIndex ? "1" : "0");
00129     for (const string& f : header_.fieldNames)
00130         ss « "," « f;
00131     for (const string& f : header_.fieldTypes)
00132         ss « "," « f;
00133     return ss.str();
00134 }
00135 /**
00136  * @brief Converts a serialized header string into a header structure.
00137  *
00138  * Parses the comma separated metadata and reconstructs the
00139  * FileHeader structure including field descriptors.
00140  *
00141  * @param s Serialized header string.
00142  * @return True if the header was successfully parsed.
00143  */
00144 bool HeaderBuffer::deserialize(const string& s) {
00145     istringstream ss(s);
00146     string tok;
00147     vector<string> parts;
00148     while (getline(ss, tok, ',')) parts.push_back(tok);
00149     if (parts.size() < 14 || (parts.size() % 2 == 1)) return false;
00150     if (parts[0] != "HDR") return false;
00151     header_.fileType          = parts[1];
00152     header_.version           = stoi(parts[2]);
00153     header_.recordSizeByteCount = stoi(parts[3]);
00154     header_.sizeFormatType     = parts[4].at(0);
00155     header_.sizeOfSizes        = stoi(parts[5]);
00156     header_.sizeIncludesItself = (parts[6] == "1");
00157     header_.indexFileName      = parts[7];
00158     header_.recordCount         = stol(parts[8]);
00159     header_.fieldCount          = stoi(parts[9]);
00160     header_.primaryKeyFieldIndex = stoi(parts[10]);
00161     header_.staleIndex          = (parts[11] == "1");
00162     header_.fieldNames.clear();
00163     header_.fieldTypes.clear();
00164     if (header_.fieldCount * 2 + 12 != parts.size()) return false;
00165     int typesStart = header_.fieldCount + 12;
00166     for (int i = 12; i < typesStart; i++)
00167         header_.fieldNames.push_back(parts[i]);
00168     for (int i = typesStart; i < (int)parts.size(); i++)
00169         header_.fieldTypes.push_back(parts[i]);
00170     return true;
00171 }
00172 /**
00173  * @brief Writes a length indicated record to the output file.
00174  * @param out Output file stream.
00175  * @param text Text content to write.
00176  * @return True if the operation succeeded.
00177  */
00178
00179 bool HeaderBuffer::writeLenLine(ofstream& out, const string& text) {
00180     out « setw(10) « setfill('0') « text.size()
00181         « ' ' « text « '\n';
00182     return static_cast<bool>(out);
00183 }
00184 /**
00185  * @brief Reads a length indicated record from the input file.
00186  *
00187  * The function first reads a fixed-width numeric field indicating
00188  * the size of the following record, then reads the record text.
00189  *
00190  * @param in Input file stream.
00191  * @param text Output variable storing the record text.
00192  * @return True if the record was successfully read.
00193  */
00194 bool HeaderBuffer::readLenLine(istream& in, string& text) {
00195     char buf[10];
00196     if (!in.read(buf, 10)) return false;
00197     for (int i = 0; i < 10; i++)
00198         if (!isdigit((unsigned char)buf[i])) return false;
00199     char sp;
00200     if (!in.get(sp) || sp != ' ') return false;
00201     int len = stoi(string(buf, 10));
00202     text.resize(len);
00203     if (!in.read(&text[0], len)) return false;
00204     if (in.peek() == '\r') in.get();
00205     if (in.peek() == '\n') in.get();
00206     return true;
00207 }

```

4.3 HeaderBuffer.h File Reference

defines the [HeaderBuffer](#) class and the [FileHeader](#) struct.

```
#include <string>
#include <vector>
#include <fstream>
```

Classes

- struct [FileHeader](#)
All header record fields required by the assignment.
- class [HeaderBuffer](#)
Reads and writes the header record for a length-indicated file.

4.3.1 Detailed Description

defines the [HeaderBuffer](#) class and the [FileHeader](#) struct.

Author

Dristi Barnwal (primary contributor)
Ethan Jackson (functional revisions and additional comments)
Marcus Julius, Teagen Lee, Natoli Mayu (reviewers)

Date

March 2026

Definition in file [HeaderBuffer.h](#).

4.4 HeaderBuffer.h

[Go to the documentation of this file.](#)

```
00001 /**
00002  * @file HeaderBuffer.h
00003  * @brief defines the HeaderBuffer class and the FileHeader struct.
00004  * @author Dristi Barnwal (primary contributor)
00005  * @author Ethan Jackson (functional revisions and additional comments)
00006  * @author Marcus Julius, Teagen Lee, Natoli Mayu (reviewers)
00007  * @date March 2026
00008  */
00009 #ifndef HEADERBUFFER_H
00010 #define HEADERBUFFER_H
00011
00012 #include <string>
00013 #include <vector>
00014 #include <fstream>
00015
00016 using namespace std;
00017
```

```

00018 /**
00019  * @struct FileHeader
00020  * @brief All header record fields required by the assignment.
00021  */
00022 struct FileHeader {
00023     bool sizeIncludesItself; ///< true if size counts itself
00024     bool staleIndex;         ///< true if index may be out of date
00025     char sizeFormatType;     ///< 'A' for ASCII, 'b' for binary
00026     int version;             ///< format version number, currently 2
00027     int sizeOfSizes;         ///< number of bytes used for record length
00028     int headerSizeBytes;     ///< size of header record in bytes
00029     int recordSizeByteCount; ///< bytes used for each record size integer
00030     int fieldCount;          ///< number of fields in each record
00031     int primaryKeyFieldIndex; ///< 0-based index of the primary key field
00032     long recordCount;        ///< total number of data records
00033     string fileType;         ///< name of file type, e.g. "ZipLenFile"
00034     string indexFileName;    ///< name of the .idx file
00035     vector<string> fieldNames; ///< the names of every field, e.g. "PlaceName"
00036     vector<string> fieldTypes; ///< the data types of every field, e.g. "int"
00037 };
00038
00039 /**
00040  * @class HeaderBuffer
00041  * @brief Reads and writes the header record for a length-indicated file.
00042  */
00043 class HeaderBuffer {
00044 public:
00045     HeaderBuffer();
00046
00047     /// Build a default header for the ZIP code file
00048     void buildDefault(const string& indexFileName, long recordCount);
00049
00050     /// Write header to an open output stream
00051     bool write(ofstream& out);
00052
00053     /// Read header from an open input stream
00054     bool read(ifstream& in);
00055
00056     /// Get the loaded header data
00057     const FileHeader& getHeader() const;
00058
00059     /// Print header contents to cout
00060     void print() const;
00061
00062 private:
00063     FileHeader header_;
00064
00065     string serialize() const;
00066     bool deserialize(const string& s);
00067     bool writeLenLine(ofstream& out, const string& text);
00068     bool readLenLine(ifstream& in, string& text);
00069 };
00070
00071 #endif

```

4.5 IndexBuilder.cpp File Reference

Implementation for building ZIP->offset index files.

```

#include "IndexBuilder.h"
#include "LenFileReader.h"
#include <fstream>
#include <iostream>
#include <string>

```

Functions

- void **buildIndex** (const string &lenFile, const string &indexFile)
Builds an index file for the provided .len file.

4.5.1 Detailed Description

Implementation for building ZIP->offset index files.

Author

Dristi Barnwal (primary contributor)

Ethan Jackson, Marcus Julius, Teagen Lee, Natoli Mayu (reviewers)

Date

March 2026

Definition in file [IndexBuilder.cpp](#).

4.5.2 Function Documentation

buildIndex()

```
void buildIndex (
    const string & lenFile,
    const string & indexFile)
```

Builds an index file for the provided .len file.

Important detail:

- We record the offset BEFORE reading the record.
- That offset points to the start of the record length field.

Parameters

<i>lenFile</i>	Input .len data file
<i>indexFile</i>	Output .idx file

Definition at line 27 of file [IndexBuilder.cpp](#).

References [readLenRecord\(\)](#).

4.6 IndexBuilder.cpp

[Go to the documentation of this file.](#)

```

00001 /**
00002  * @file IndexBuilder.cpp
00003  * @brief Implementation for building ZIP->offset index files.
00004  * @author Dristi Barnwal (primary contributor)
00005  * @author Ethan Jackson, Marcus Julius, Teagen Lee, Natoli Mayu (reviewers)
00006  * @date March 2026
00007  */
00008 #include "IndexBuilder.h"
00009 #include "LenFileReader.h"
00010
00011 #include <fstream>
00012 #include <iostream>
00013 #include <string>
00014
00015 using namespace std;
00016
00017 /**
00018  * @brief Builds an index file for the provided .len file.
00019  *
00020  * Important detail:
00021  * - We record the offset BEFORE reading the record.
00022  * - That offset points to the start of the record length field.
00023  *
00024  * @param lenFile Input .len data file
00025  * @param indexFile Output .idx file
00026  */
00027 void buildIndex(const string& lenFile, const string& indexFile)
00028 {
00029     ifstream in(lenFile);
00030     if (!in) {
00031         cout << "Error: Cannot open LEN file: " << lenFile << "\n";
00032         return;
00033     }
00034
00035     ofstream out(indexFile);
00036     if (!out) {
00037         cout << "Error: Cannot create index file: " << indexFile << "\n";
00038         return;
00039     }
00040
00041     // Read the LEN header record first (we skip it for indexing records).
00042     string header;
00043     if (!readLenRecord(in, header)) {
00044         cout << "Error: LEN file has no header or bad format.\n";
00045         return;
00046     }
00047
00048     // Optional index file header (helps identify it)
00049     out << "IDX,1\n";
00050
00051     long long count = 0;
00052
00053     while (true)
00054     {
00055         // Save the start offset of the next record
00056         streampos pos = in.tellg();
00057
00058         // Read record text
00059         string record;
00060         if (!readLenRecord(in, record))
00061             break; // reached EOF
00062
00063         // ZIP is the first field before comma
00064         size_t comma = record.find(',');
00065         if (comma == string::npos) {
00066             // Bad record: skip it
00067             continue;
00068         }
00069
00070         string zip = record.substr(0, comma); // keep leading zeros if any
00071
00072         // Write: ZIP offset
00073         out << zip << " " << static_cast<long long>(pos) << "\n";
00074         count++;
00075     }
00076

```

```
00077     cout << "Index created: " << indexFile << " (entries=" << count << ")\n";
00078 }
```

4.7 IndexBuilder.h File Reference

Builds a primary key index file for a .len data file.

```
#include <string>
```

Functions

- void [buildIndex](#) (const std::string &lenFile, const std::string &indexFile)
Builds an index file from a .len file.

4.7.1 Detailed Description

Builds a primary key index file for a .len data file.

Author

Dristi Barnwal (primary contributor)

Ethan Jackson, Marcus Julius, Teagen Lee, Natoli Mayu (reviewers)

Date

March 2026

The index file stores:

- ZIP code (primary key)
- byte offset (file position) in the .len file where the record starts

Example index line: 56301 120

Meaning:

- ZIP 56301 starts at byte 120 in the data file.

Why we do this:

- During search, we load the index into RAM (allowed).
- Then we can jump directly to a ZIP record using `seekg(offset)`.
- We read only ONE record at a time (RAM rule).

Definition in file [IndexBuilder.h](#).

4.7.2 Function Documentation

buildIndex()

```
void buildIndex (
    const std::string & lenFile,
    const std::string & indexFile)
```

Builds an index file from a .len file.

This function: 1) Opens the .len data file 2) Reads and skips the LEN header record 3) For each record:

- saves the current file position (offset)
- reads the record
- extracts ZIP (first field before comma)
- writes ZIP and offset into the index file

Parameters

<i>lenFile</i>	Path to the length-indicated data file
<i>indexFile</i>	Path to the output index file

4.8 IndexBuilder.h

[Go to the documentation of this file.](#)

```
00001 /**
00002  * @file IndexBuilder.h
00003  * @brief Builds a primary key index file for a .len data file.
00004  * @author Dristi Barnwal (primary contributor)
00005  * @author Ethan Jackson, Marcus Julius, Teagen Lee, Natoli Mayu (reviewers)
00006  * @date March 2026
00007  *
00008  * The index file stores:
00009  * - ZIP code (primary key)
00010  * - byte offset (file position) in the .len file where the record starts
00011  *
00012  * Example index line:
00013  * 56301 120
00014  *
00015  * Meaning:
00016  * - ZIP 56301 starts at byte 120 in the data file.
00017  *
00018  * Why we do this:
00019  * - During search, we load the index into RAM (allowed).
00020  * - Then we can jump directly to a ZIP record using seekg(offset).
00021  * - We read only ONE record at a time (RAM rule).
00022  */
00023 #ifndef INDEXBUILDER_H
00024 #define INDEXBUILDER_H
00025
00026 #include <string>
00027
00028 /**
00029  * @brief Builds an index file from a .len file.
00030  *
00031  * This function:
00032  * 1) Opens the .len data file
00033  * 2) Reads and skips the LEN header record
00034  * 3) For each record:
```



```

00035 *      - saves the current file position (offset)
00036 *      - reads the record
00037 *      - extracts ZIP (first field before comma)
00038 *      - writes ZIP and offset into the index file
00039 *
00040 * @param lenFile Path to the length-indicated data file
00041 * @param indexFile Path to the output index file
00042 */
00043 void buildIndex(const std::string& lenFile, const std::string& indexFile);
00044
00045 #endif

```

4.9 LenFileReader.cpp File Reference

Implementation for reading length-indicated records.

```

#include "LenFileReader.h"
#include <cctype>

```

Functions

- bool [readLenRecord](#)(ifstream &in, string &recordLine)
Reads one length-indicated record from the input stream.

4.9.1 Detailed Description

Implementation for reading length-indicated records.

Author

Dristi Barnwal (primary contributor)
Ethan Jackson, Marcus Julius, Teagen Lee, Natoli Mayu (reviewers)

Date

March 2026

Definition in file [LenFileReader.cpp](#).

4.9.2 Function Documentation

readLenRecord()

```

bool readLenRecord (
    ifstream & in,
    string & recordLine)

```

Reads one length-indicated record from the input stream.

This is the key function that makes the data file "seekable". Once we also build an index (ZIP -> offset), we can do:

- seekg(offset)
- readLenRecord(...) and get exactly one ZIP record.

Parameters

<i>in</i>	Input stream
-----------	--------------

<i>recordLine</i>	Output record data (CSV-style text)
-------------------	-------------------------------------

Returns

true if successful, false if EOF or bad format

Definition at line 26 of file [LenFileReader.cpp](#).

Referenced by [buildIndex\(\)](#).

4.10 LenFileReader.cpp

[Go to the documentation of this file.](#)

```

00001 /**
00002  * @file LenFileReader.cpp
00003  * @brief Implementation for reading length-indicated records.
00004  * @author Dristi Barnwal (primary contributor)
00005  * @author Ethan Jackson, Marcus Julius, Teagen Lee, Natoli Mayu (reviewers)
00006  * @date March 2026
00007  */
00008 #include "LenFileReader.h"
00009 #include <cctype>    // for std::isdigit
00010
00011 using namespace std;
00012
00013 /**
00014  * @brief Reads one length-indicated record from the input stream.
00015  *
00016  * This is the key function that makes the data file "seekable".
00017  * Once we also build an index (ZIP -> offset), we can do:
00018  * - seekg(offset)
00019  * - readLenRecord(...)
00020  * and get exactly one ZIP record.
00021  *
00022  * @param in Input stream
00023  * @param recordLine Output record data (CSV-style text)
00024  * @return true if successful, false if EOF or bad format
00025  */
00026 bool readLenRecord(istream& in, string& recordLine)
00027 {
00028     recordLine.clear();
00029
00030     // 1) Read 10 bytes for length
00031     char lenBuf[10];
00032     if (!in.read(lenBuf, 10)) {
00033         // likely EOF or stream error
00034         return false;
00035     }
00036
00037     // 2) Validate they are digits (simple safety check)
00038     for (int i = 0; i < 10; i++) {
00039         if (!isdigit(static_cast<unsigned char>(lenBuf[i]))) {
00040             return false; // bad format
00041         }
00042     }
00043
00044     // 3) Read the space after length
00045     char space;
00046     if (!in.get(space) || space != ' ') {
00047         return false; // bad format
00048     }
00049
00050     // Convert length buffer into integer
00051     int length = stoi(string(lenBuf, 10));
00052     if (length < 0) return false;
00053
00054     // 4) Read exactly "length" characters of record data
00055     recordLine.resize(length);

```

```

00056     if (!in.read(&recordLine[0], length)) {
00057         return false; // truncated file
00058     }
00059
00060     // 5) Consume newline if present (supports \n or \r\n)
00061     if (in.peek() == '\n') {
00062         in.get();
00063     } else if (in.peek() == '\r') {
00064         in.get();
00065         if (in.peek() == '\n') in.get();
00066     }
00067
00068     return true;
00069 }

```

4.11 LenFileReader.h File Reference

Reads one record at a time from a length-indicated file (.len).

```

#include <string>
#include <fstream>

```

Functions

- bool [readLenRecord](#) (std::ifstream &in, std::string &recordLine)
Reads one length-indicated record from a .len file.

4.11.1 Detailed Description

Reads one record at a time from a length-indicated file (.len).

Author

Dristi Barnwal (primary contributor)
 Ethan Jackson, Marcus Julius, Teagen Lee, Natoli Mayu

The LEN file format for each record is: [10 ASCII digits length][space][record text][newline]

Example: 0000000042 56301,St Cloud,MN,Stearns,45.5579,-94.1632

This reader helps the program follow the RAM rule:

- Only ONE record is read at a time.
- The entire data file is not loaded into memory.

Definition in file [LenFileReader.h](#).

4.11.2 Function Documentation

readLenRecord()

```
bool readLenRecord (
    std::ifstream & in,
    std::string & recordLine)
```

Reads one length-indicated record from a .len file.

Steps: 1) Read 10 characters for the length 2) Convert them to an integer 3) Read 1 space character 4) Read exactly "length" characters into recordLine 5) Optionally consume newline at the end

Parameters

<i>in</i>	Input file stream (must already be open)
-----------	--

<i>recordLine</i>	Output string that receives the record data text
-------------------	--

Returns

true if a record was read successfully, false if EOF or format error

4.12 LenFileReader.h

[Go to the documentation of this file.](#)

```

00001 /**
00002  * @file LenFileReader.h
00003  * @brief Reads one record at a time from a length-indicated file (.len).
00004  * @author Dristi Barnwal (primary contributor)
00005  * @author Ethan Jackson, Marcus Julius, Teagen Lee, Natoli Mayu
00006  *
00007  * The LEN file format for each record is:
00008  * [10 ASCII digits length][space][record text][newline]
00009  *
00010  * Example:
00011  * 0000000042 56301,St Cloud,MN,Stearns,45.5579,-94.1632
00012  *
00013  * This reader helps the program follow the RAM rule:
00014  * - Only ONE record is read at a time.
00015  * - The entire data file is not loaded into memory.
00016  */
00017 #ifndef LENFILEREADER_H
00018 #define LENFILEREADER_H
00019
00020 #include <string>
00021 #include <fstream>
00022
00023 /**
00024  * @brief Reads one length-indicated record from a .len file.
00025  *
00026  * Steps:
00027  * 1) Read 10 characters for the length
00028  * 2) Convert them to an integer
00029  * 3) Read 1 space character
00030  * 4) Read exactly "length" characters into recordLine
00031  * 5) Optionally consume newline at the end
00032  *
00033  * @param in Input file stream (must already be open)
00034  * @param recordLine Output string that receives the record data text
00035  * @return true if a record was read successfully, false if EOF or format error
00036  */
00037 bool readLenRecord(std::ifstream& in, std::string& recordLine);
00038
00039 #endif

```

4.13 LenFileWriter.cpp File Reference

Implementation of CSV -> .len conversion.

```

#include "LenFileWriter.h"
#include <fstream>
#include <iostream>
#include <iomanip>

```

Functions

- static bool [writeLenLine](#) (ostream &out, const string &text)
Writes one length-indicated line to an output stream.
- void [makeLenFile](#) (const string &csvFile, const string &lenFile)
Converts a CSV file into a length-indicated (.len) file.

4.13.1 Detailed Description

Implementation of CSV -> .len conversion.

Author

Dristi Barnwal (primary contributor)
Ethan Jackson, Marcus Julius, Teagen Lee, Natoli Mayu (reviewers)

Date

March 2026

This file contains the code that writes the length-indicated format.

Definition in file [LenFileWriter.cpp](#).

4.13.2 Function Documentation

makeLenFile()

```
void makeLenFile (  
    const string & csvFile,  
    const string & lenFile)
```

Converts a CSV file into a length-indicated (.len) file.

Simple steps: 1) Open input CSV file for reading 2) Open output LEN file for writing 3) Skip the first CSV line (header) 4) Write a LEN header record (also length-indicated) 5) For every remaining CSV line, write it as a LEN record

Parameters

<i>csvFile</i>	Input CSV filename
<i>lenFile</i>	Output LEN filename

Definition at line [56](#) of file [LenFileWriter.cpp](#).

References [writeLenLine\(\)](#).

writeLenLine()

```
bool writeLenLine (  
    ostream & out,  
    const string & text) [static]
```

Writes one length-indicated line to an output stream.

The output format is: [10 digits length][space][text][newline]

Example: 0000000005 hello

Parameters

<i>out</i>	Output stream (file)
------------	----------------------

<code>text</code>	The text to write (this is the record data)
-------------------	---

Returns

true if writing succeeded, false otherwise

Definition at line 30 of file [LenFileWriter.cpp](#).

Referenced by [makeLenFile\(\)](#).

4.14 LenFileWriter.cpp

[Go to the documentation of this file.](#)

```

00001 /**
00002  * @file LenFileWriter.cpp
00003  * @brief Implementation of CSV -> .len conversion.
00004  * @author Dristi Barnwal (primary contributor)
00005  * @author Ethan Jackson, Marcus Julius, Teagen Lee, Natoli Mayu (reviewers)
00006  * @date March 2026
00007  *
00008  * This file contains the code that writes the length-indicated format.
00009  */
00010 #include "LenFileWriter.h"
00011 #include <fstream>
00012 #include <iostream>
00013 #include <iomanip>
00014
00015 using namespace std;
00016
00017 /**
00018  * @brief Writes one length-indicated line to an output stream.
00019  *
00020  * The output format is:
00021  * [10 digits length][space][text][newline]
00022  *
00023  * Example:
00024  * 0000000005 hello
00025  *
00026  * @param out Output stream (file)
00027  * @param text The text to write (this is the record data)
00028  * @return true if writing succeeded, false otherwise
00029  */
00030 static bool writeLenLine(ostream& out, const string& text) {
00031     // If the record text is empty, we choose not to write it.
00032     if (text.empty()) return false;
00033
00034     // Write length as 10 digits with leading zeros.
00035     // Example: 42 becomes 0000000042
00036     out << setw(10) << setfill('0') << text.size()
00037         << ' ' << text << '\n';
00038
00039     // Return whether the stream is still good.
00040     return static_cast<bool>(out);
00041 }
00042
00043 /**
00044  * @brief Converts a CSV file into a length-indicated (.len) file.
00045  *
00046  * Simple steps:
00047  * 1) Open input CSV file for reading
00048  * 2) Open output LEN file for writing
00049  * 3) Skip the first CSV line (header)
00050  * 4) Write a LEN header record (also length-indicated)
00051  * 5) For every remaining CSV line, write it as a LEN record
00052  *
00053  * @param csvFile Input CSV filename
00054  * @param lenFile Output LEN filename
00055  */

```



```

00056 void makeLenFile(const string& csvFile, const string& lenFile)
00057 {
00058     ifstream in(csvFile);
00059     if (!in) {
00060         cout << "Error: Cannot open CSV file: " << csvFile << "\n";
00061         return;
00062     }
00063
00064     ofstream out(lenFile);
00065     if (!out) {
00066         cout << "Error: Cannot create LEN file: " << lenFile << "\n";
00067         return;
00068     }
00069
00070     // Read and ignore the CSV header row (field names line).
00071     string csvHeader;
00072     if (!getline(in, csvHeader)) {
00073         cout << "Error: CSV file is empty: " << csvFile << "\n";
00074         return;
00075     }
00076
00077     // Write a simple LEN header record (you can expand it later).
00078     // This record itself also uses the LEN format.
00079     // Example header text: HDR,ZipLenFile,1,SIZEFMT=ASCII,SIZEWIDTH=10
00080     string lenHeaderText = "HDR,ZipLenFile,1,SIZEFMT=ASCII,SIZEWIDTH=10";
00081     if (!writeLenLine(out, lenHeaderText)) {
00082         cout << "Error: Failed writing LEN header record.\n";
00083         return;
00084     }
00085
00086     long long count = 0;
00087
00088     // Read each CSV record line and write it into LEN file.
00089     string line;
00090     while (getline(in, line)) {
00091         if (line.empty()) continue; // skip empty lines
00092
00093         if (!writeLenLine(out, line)) {
00094             cout << "Warning: Skipped a line that could not be written.\n";
00095             continue;
00096         }
00097         count++;
00098     }
00099
00100     cout << "LEN file created: " << lenFile << " (records=" << count << ")\n";
00101 }

```

4.15 LenFileWriter.h File Reference

Converts a CSV file into a length-indicated file (.len).

```
#include <string>
```

Functions

- void `makeLenFile` (const std::string &csvFile, const std::string &lenFile)
Creates a .len file from a CSV file.

4.15.1 Detailed Description

Converts a CSV file into a length-indicated file (.len).

Author

Dristi Barnwal (primary contributor)

Ethan Jackson, Marcus Julius, Teagen Lee, Natoli Mayu (reviewers)

Date

March 2026

A "length-indicated" file means:

- Each record (line) is saved with its size (length) written first.
- Then the actual CSV record is written.

Example of one record in a .len file: 0000000042 56301,St Cloud,MN,Stearns,45.5579,-94.1632

Here:

- "0000000042" is 10 digits (fixed width).
- It tells how many characters are in the CSV text after the space.
- The CSV text is still comma-separated.

Why we do this:

- Later, we will build an index that stores ZIP -> file position (offset).
- Then we can jump directly to a ZIP record without reading the whole file.

Definition in file [LenFileWriter.h](#).

4.15.2 Function Documentation

makeLenFile()

```
void makeLenFile (  
    const std::string & csvFile,  
    const std::string & lenFile)
```

Creates a .len file from a CSV file.

This function: 1) Opens the input CSV file 2) Skips the first line (CSV header row) 3) Writes a header record into the .len output file 4) For each CSV record line:

- writes 10-digit ASCII length
- writes one space
- writes the CSV record text
- writes newline

Parameters

<i>csvFile</i>	Path to the input CSV file (e.g. "us_postal_codes.csv")
----------------	---

<i>lenFile</i>	Path to the output length-indicated file (e.g. "sorted.len")
----------------	--

4.16 LenFileWriter.h

[Go to the documentation of this file.](#)

```

00001 /**
00002  * @file LenFileWriter.h
00003  * @brief Converts a CSV file into a length-indicated file (.len).
00004  * @author Dristi Barnwal (primary contributor)
00005  * @author Ethan Jackson, Marcus Julius, Teagen Lee, Natoli Mayu (reviewers)
00006  * @date March 2026
00007  *
00008  * A "length-indicated" file means:
00009  * - Each record (line) is saved with its size (length) written first.
00010  * - Then the actual CSV record is written.
00011  *
00012  * Example of one record in a .len file:
00013  * 00000000042 56301,St Cloud,MN,Stearns,45.5579,-94.1632
00014  *
00015  * Here:
00016  * - "00000000042" is 10 digits (fixed width).
00017  * - It tells how many characters are in the CSV text after the space.
00018  * - The CSV text is still comma-separated.
00019  *
00020  * Why we do this:
00021  * - Later, we will build an index that stores ZIP -> file position (offset).
00022  * - Then we can jump directly to a ZIP record without reading the whole file.
00023  */
00024 #ifndef LENFILEWRITER_H
00025 #define LENFILEWRITER_H
00026
00027 #include <string>
00028
00029 /**
00030  * @brief Creates a .len file from a CSV file.
00031  *
00032  * This function:
00033  * 1) Opens the input CSV file
00034  * 2) Skips the first line (CSV header row)
00035  * 3) Writes a header record into the .len output file
00036  * 4) For each CSV record line:
00037  *    - writes 10-digit ASCII length
00038  *    - writes one space
00039  *    - writes the CSV record text
00040  *    - writes newline
00041  *
00042  * @param csvFile Path to the input CSV file (e.g. "us_postal_codes.csv")
00043  * @param lenFile Path to the output length-indicated file (e.g. "sorted.len")
00044  */
00045 void makeLenFile(const std::string& csvFile,
00046                 const std::string& lenFile);
00047
00048 #endif

```

4.17 main.cpp File Reference

Zip Code Group Project 2.0 main controller (CSV → LEN, Index, Search, Analyze).

```

#include "ZipCodeBuffer.h"
#include "HeaderBuffer.h"
#include <iostream>
#include <fstream>
#include <sstream>
#include <string>
#include <vector>
#include <map>

```

```
#include <unordered_map>
#include <iomanip>
#include <limits>
#include <cctype>
```

Classes

- struct [StateExtremes](#)
Keeps the most extreme ZIP codes for one state (east/west/north/south).

Functions

- static bool [smallerZipWins](#) (int candidate, int current)
If coordinate ties, choose the smaller ZIP.
- static void [updateStateExtremes](#) (map< string, [StateExtremes](#) > &stateMap, const [ZipCodeRecord](#) &record)
Update state extremes using one record.
- static void [printStateExtremesTable](#) (const map< string, [StateExtremes](#) > &stateMap)
Print the state extremes table (same idea as Project 1).
- static bool [writeLenLine](#) (ostream &out, const string &text)
Write one "length-indicated" record to an output stream.
- static bool [readLenLine](#) (istream &in, string &textOut)
Read one "length-indicated" record from an input stream.
- static vector< string > [splitCsvSimple](#) (const string &line)
Split a CSV line into fields (basic quote support).
- static void [printLabeledOneLine](#) (const string &csvLine)
Print one record with labels on ONE line (Part II requirement).
- static int [analyzeCsvStreaming](#) (const string &csvFile)
Analyze state extremes from a CSV file without loading all records.
- static int [makeLenFromCsv](#) (const string &csvFile, const string &lenFile)
Convert CSV → LEN file.
- static int [buildIndexFromLen](#) (const string &lenFile, const string &idxFile)
Build a primary key index (ZIP → byte offset) from a LEN file.
- static unordered_map< string, streampos > [loadIndex](#) (const string &idxFile)
Load an index file into RAM.
- static int [searchZips](#) (const string &lenFile, const string &idxFile, const vector< string > &zips)
Search all -Z flags provided and print results.
- static void [printUsage](#) (const string &prog)
Print usage instructions (simple).
- int [main](#) (int argc, char *argv[])

4.17.1 Detailed Description

Zip Code Group Project 2.0 main controller (CSV → LEN, Index, Search, Analyze).

This is a "multi-mode" program. It can run different tasks depending on the command.

4.17.1.1 MODES (commands you can run)

- 1) Project 1 style CSV analysis (streaming / no vector) `./zipprog <csv_file>`
- 2) Convert CSV → length-indicated data file (.len) `./zipprog -make-len <input.csv> <output.len>`
- 3) Build primary-key index from .len `./zipprog -build-index <data.len> <index.idx>`
- 4) Search ZIP(s) using index (flags like -Z56301) `./zipprog -search <data.len> <index.idx> -Z56301 -Z99546 -Z99999`

Notes:

- For Project 2 RAM rule during searching: We only keep: (1) index in RAM (2) one record string / one [ZipCodeRecord](#) at a time
- This file uses the fixed-width ASCII length format: [10 digits][space][recordText][newline]

Author

Dristi Barnwal

Date

March 2026

Definition in file [main.cpp](#).

4.17.2 Function Documentation

analyzeCsvStreaming()

```
int analyzeCsvStreaming (
    const string & csvFile) [static]
```

Analyze state extremes from a CSV file without loading all records.

Parameters

<i>csvFile</i>	Input CSV filename
----------------	--------------------

Returns

exit code (0 success)

Definition at line 324 of file [main.cpp](#).

References [ZipCodeBuffer::close\(\)](#), [ZipCodeBuffer::open\(\)](#), [printStatsExtremesTable\(\)](#), [ZipCodeBuffer::readRecord\(\)](#), and [updateStateExtremes\(\)](#).

Referenced by [main\(\)](#).

buildIndexFromLen()

```
int buildIndexFromLen (
    const string & lenFile,
    const string & idxFile) [static]
```

Build a primary key index (ZIP → byte offset) from a LEN file.

We store the file position BEFORE reading each record. That position is the “start of record” (the 10-digit length field).

Index file format (simple): IDX,1 56301 128 02139 412

Parameters

<i>lenFile</i>	Input LEN data file
----------------	---------------------

<i>idxFile</i>	Output index file
----------------	-------------------

Returns

exit code

Definition at line [436](#) of file [main.cpp](#).

References [readLenLine\(\)](#).

Referenced by [main\(\)](#).

loadIndex()

```
unordered_map< string, streampos > loadIndex (  
    const string & idxFile) [static]
```

Load an index file into RAM.

Parameters

<i>idxFile</i>	Index filename
----------------	----------------

Returns

map ZIP(string) → offset

Definition at line [488](#) of file [main.cpp](#).

Referenced by [searchZips\(\)](#).

main()

```
int main (  
    int argc,  
    char * argv[])
```

Definition at line [586](#) of file [main.cpp](#).

References [analyzeCsvStreaming\(\)](#), [buildIndexFromLen\(\)](#), [makeLenFromCsv\(\)](#), [printUsage\(\)](#), and [searchZips\(\)](#).

makeLenFromCsv()

```
int makeLenFromCsv (  
    const string & csvFile,  
    const string & lenFile) [static]
```

Convert CSV → LEN file.

We skip the CSV header row, then write: 1) a LEN header record (simple placeholder text) 2) each CSV record as a length-indicated record

Parameters

<i>csvFile</i>	Input CSV
<i>lenFile</i>	Output LEN

Returns

exit code

Definition at line 372 of file [main.cpp](#).

References [HeaderBuffer::buildDefault\(\)](#), [HeaderBuffer::write\(\)](#), and [writeLenLine\(\)](#).

Referenced by [main\(\)](#).

printLabeledOneLine()

```
void printLabeledOneLine (
    const string & csvLine) [static]
```

Print one record with labels on ONE line (Part II requirement).

Parameters

<i>csvLine</i>	The record data (CSV text inside LEN)
----------------	---------------------------------------

Expected order (for now): ZipCode,PlaceName,State,County,Lat,Long

If you later support column re-ordering using header metadata, you will change this to print by field names from the header mapping.

Definition at line 286 of file [main.cpp](#).

References [splitCsvSimple\(\)](#).

Referenced by [searchZips\(\)](#).

printStateExtremesTable()

```
void printStateExtremesTable (
    const map< string, StateExtremes > & stateMap) [static]
```

Print the state extremes table (same idea as Project 1).

Parameters

<i>stateMap</i>	Map of state → extremes
-----------------	-------------------------

Definition at line 149 of file [main.cpp](#).

References [StateExtremes::easternmost](#), [StateExtremes::northernmost](#), [StateExtremes::southernmost](#), and [StateExtremes::westernmost](#).

Referenced by [analyzeCsvStreaming\(\)](#).

printUsage()

```
void printUsage (
    const string & prog) [static]
```

Print usage instructions (simple).

Parameters

<i>prog</i>	program name
-------------	--------------

Definition at line 569 of file [main.cpp](#).

Referenced by [main\(\)](#).

readLenLine()

```
bool readLenLine (
```

```
istream & in,  
string & textOut) [static]
```

Read one "length-indicated" record from an input stream.

Steps: 1) Read 10 chars (must be digits) 2) Read one space 3) Read exactly <length> chars into textOut 4) Consume newline if present

Parameters

<i>in</i>	Input stream
<i>textOut</i>	Output record text (CSV-like)

Returns

true if record was read, false if EOF or format error

Definition at line 221 of file [main.cpp](#).

Referenced by [buildIndexFromLen\(\)](#), and [searchZips\(\)](#).

searchZips()

```
int searchZips (  
    const string & lenFile,  
    const string & idxFile,  
    const vector< string > & zips) [static]
```

Search all -Z flags provided and print results.

Parameters

<i>lenFile</i>	Data file (.len)
<i>idxFile</i>	Index file (.idx)
<i>zips</i>	List of ZIP strings to search

Returns

exit code

Definition at line 512 of file [main.cpp](#).

References [loadIndex\(\)](#), [printLabeledOneLine\(\)](#), and [readLenLine\(\)](#).

Referenced by [main\(\)](#).

smallerZipWins()

```
bool smallerZipWins (  
    int candidate,  
    int current) [static]
```

If coordinate ties, choose the smaller ZIP.

Parameters

<i>candidate</i>	ZIP we are considering
<i>current</i>	ZIP already stored

Returns

true if candidate should replace current

Definition at line 93 of file [main.cpp](#).

Referenced by [updateStateExtremes\(\)](#).

splitCsvSimple()

```
vector< string > splitCsvSimple (
    const string & line) [static]
```

Split a CSV line into fields (basic quote support).

Parameters

<i>line</i>	CSV line
-------------	----------

Returns

vector of fields

Definition at line 257 of file [main.cpp](#).

Referenced by [printLabeledOneLine\(\)](#).

updateStateExtremes()

```
void updateStateExtremes (
    map< string, StateExtremes > & stateMap,
    const ZipCodeRecord & record) [static]
```

Update state extremes using one record.

Parameters

<i>stateMap</i>	Map of state → extremes (updates inside)
<i>record</i>	One ZIP record

Definition at line 103 of file [main.cpp](#).

References [StateExtremes::easternmost](#), [ZipCodeRecord::latitude](#), [ZipCodeRecord::longitude](#), [StateExtremes::maxLatitude](#), [StateExtremes::maxLongitude](#), [StateExtremes::minLatitude](#), [StateExtremes::minLongitude](#), [StateExtremes::northernmost](#), [smallerZipWins\(\)](#), [StateExtremes::southernmost](#), [ZipCodeRecord::state](#), [StateExtremes::westernmost](#), and [ZipCodeRecord::zipCode](#).

Referenced by [analyzeCsvStreaming\(\)](#).

writeLenLine()

```
bool writeLenLine (
    ostream & out,
    const string & text) [static]
```

Write one "length-indicated" record to an output stream.

Format: [10 digits length][space][text][newline]

Example: 0000000042 56301,St Cloud,MN,Stearns,45.5579,-94.1632

Parameters

<i>out</i>	Output stream
------------	---------------

<i>text</i>	Record text (still CSV inside)
-------------	--------------------------------

Returns

true if write succeeded

Definition at line 199 of file `main.cpp`.

Referenced by `makeLenFromCsv()`.

4.18 main.cpp

[Go to the documentation of this file.](#)

```

00001 /**
00002  * @file main.cpp
00003  * @brief Zip Code Group Project 2.0 main controller (CSV → LEN, Index, Search, Analyze)
00004  *
00005  * This is a "multi-mode" program. It can run different tasks depending on the command.
00006  *
00007  * -----
00008  * MODES (commands you can run)
00009  * -----
00010  *
00011  * 1) Project 1 style CSV analysis (streaming / no vector)
00012  *    ./zipprog <csv_file>
00013  *
00014  * 2) Convert CSV → length-indicated data file (.len)
00015  *    ./zipprog --make-len <input.csv> <output.len>
00016  *
00017  * 3) Build primary-key index from .len
00018  *    ./zipprog --build-index <data.len> <index.idx>
00019  *
00020  * 4) Search ZIP(s) using index (flags like -Z56301)
00021  *    ./zipprog --search <data.len> <index.idx> -Z56301 -Z99546 -Z99999
00022  *
00023  * Notes:
00024  * - For Project 2 RAM rule during searching:
00025  *   We only keep:
00026  *     (1) index in RAM
00027  *     (2) one record string / one ZipCodeRecord at a time
00028  * - This file uses the fixed-width ASCII length format:
00029  *   [10 digits][space][recordText][newline]
00030  *
00031  * @author Dristi Barnwal
00032  * @date March 2026
00033  */
00034
00035 #include "ZipCodeBuffer.h"
00036 #include "HeaderBuffer.h"
00037
00038 #include <iostream>
00039 #include <fstream>
00040 #include <sstream>
00041 #include <string>
00042 #include <vector>
00043 #include <map>
00044 #include <unordered_map>
00045 #include <iomanip>
00046 #include <limits>
00047 #include <cctype>
00048
00049 using namespace std;
00050
00051 /* =====
00052  * STRUCTS USED FOR PROJECT 1 ANALYSIS (STATE EXTREMES)
00053  * =====
00054  */
00055
00056 /**
00057  * @struct StateExtremes
00058  * @brief Keeps the most extreme ZIP codes for one state (east/west/north/south).
00059  *
00060  * We store:
00061  * - Which ZIP code is easternmost / westernmost / northernmost / southernmost
00062  * - The coordinate values used to compare

```

```

00063  *
00064  * Tie-breaking rule:
00065  * - If two ZIPs tie on coordinate, choose the smaller ZIP so results are stable
00066  *   even if the input rows are shuffled.
00067  */
00068  struct StateExtremes {
00069      int easternmost;
00070      int westernmost;
00071      int northernmost;
00072      int southernmost;
00073
00074      double minLongitude;
00075      double maxLongitude;
00076      double maxLatitude;
00077      double minLatitude;
00078
00079      StateExtremes()
00080          : easternmost(0), westernmost(0), northernmost(0), southernmost(0),
00081            minLongitude(numeric_limits<double>::max()),
00082            maxLongitude(numeric_limits<double>::lowest()),
00083            maxLatitude(numeric_limits<double>::lowest()),
00084            minLatitude(numeric_limits<double>::max()) {}
00085  };
00086
00087  /**
00088   * @brief If coordinate ties, choose the smaller ZIP.
00089   * @param candidate ZIP we are considering
00090   * @param current ZIP already stored
00091   * @return true if candidate should replace current
00092   */
00093  static bool smallerZipWins(int candidate, int current) {
00094      if (current == 0) return true; // current not set yet
00095      return candidate < current;
00096  }
00097
00098  /**
00099   * @brief Update state extremes using one record.
00100   * @param stateMap Map of state → extremes (updates inside)
00101   * @param record One ZIP record
00102   */
00103  static void updateStateExtremes(map<string, StateExtremes>& stateMap,
00104                                  const ZipCodeRecord& record) {
00105      const string& state = record.state;
00106      StateExtremes& ex = stateMap[state]; // creates entry if missing
00107
00108      // EASTERNMOST (min longitude)
00109      if (record.longitude < ex.minLongitude) {
00110          ex.minLongitude = record.longitude;
00111          ex.easternmost = record.zipCode;
00112      } else if (record.longitude == ex.minLongitude &&
00113                  smallerZipWins(record.zipCode, ex.easternmost)) {
00114          ex.easternmost = record.zipCode;
00115      }
00116
00117      // WESTERNMOST (max longitude)
00118      if (record.longitude > ex.maxLongitude) {
00119          ex.maxLongitude = record.longitude;
00120          ex.westernmost = record.zipCode;
00121      } else if (record.longitude == ex.maxLongitude &&
00122                  smallerZipWins(record.zipCode, ex.westernmost)) {
00123          ex.westernmost = record.zipCode;
00124      }
00125
00126      // NORTHERNMOST (max latitude)
00127      if (record.latitude > ex.maxLatitude) {
00128          ex.maxLatitude = record.latitude;
00129          ex.northernmost = record.zipCode;
00130      } else if (record.latitude == ex.maxLatitude &&
00131                  smallerZipWins(record.zipCode, ex.northernmost)) {
00132          ex.northernmost = record.zipCode;
00133      }
00134
00135      // SOUTHERNMOST (min latitude)
00136      if (record.latitude < ex.minLatitude) {
00137          ex.minLatitude = record.latitude;
00138          ex.southernmost = record.zipCode;
00139      } else if (record.latitude == ex.minLatitude &&
00140                  smallerZipWins(record.zipCode, ex.southernmost)) {
00141          ex.southernmost = record.zipCode;
00142      }
00143  }

```

```

00144
00145 /**
00146  * @brief Print the state extremes table (same idea as Project 1).
00147  * @param stateMap Map of state → extremes
00148  */
00149 static void printStateExtremesTable(const map<string, StateExtremes>& stateMap) {
00150     cout << left;
00151     cout << setw(8) << "State"
00152           << setw(15) << "Easternmost"
00153           << setw(15) << "Westernmost"
00154           << setw(15) << "Northernmost"
00155           << setw(15) << "Southernmost"
00156           << "\n";
00157     cout << string(68, '-') << "\n";
00158
00159     for (const auto& entry : stateMap) {
00160         const string& state = entry.first;
00161         const StateExtremes& ex = entry.second;
00162
00163         cout << setw(8) << state;
00164
00165         cout << setfill('0') << setw(5) << ex.easternmost
00166               << setfill(' ') << setw(10) << " ";
00167
00168         cout << setfill('0') << setw(5) << ex.westernmost
00169               << setfill(' ') << setw(10) << " ";
00170
00171         cout << setfill('0') << setw(5) << ex.northernmost
00172               << setfill(' ') << setw(10) << " ";
00173
00174         cout << setfill('0') << setw(5) << ex.southernmost
00175               << setfill(' ') << "\n";
00176     }
00177
00178     cout << "\nTotal states/territories: " << stateMap.size() << "\n";
00179 }
00180
00181 /* =====
00182  * LENGTH-INDICATED FILE HELPERS (ASCII fixed-width length = 10 digits)
00183  * =====
00184  */
00185
00186 /**
00187  * @brief Write one "length-indicated" record to an output stream.
00188  *
00189  * Format:
00190  * [10 digits length][space][text][newline]
00191  *
00192  * Example:
00193  * 0000000042 56301,St Cloud,MN,Stearns,45.5579,-94.1632
00194  *
00195  * @param out Output stream
00196  * @param text Record text (still CSV inside)
00197  * @return true if write succeeded
00198  */
00199 static bool writeLenLine(ostream& out, const string& text) {
00200     if (text.empty()) return false;
00201
00202     out << setw(10) << setfill('0') << text.size()
00203           << ' ' << text << '\n';
00204
00205     return static_cast<bool>(out);
00206 }
00207
00208 /**
00209  * @brief Read one "length-indicated" record from an input stream.
00210  *
00211  * Steps:
00212  * 1) Read 10 chars (must be digits)
00213  * 2) Read one space
00214  * 3) Read exactly <length> chars into textOut
00215  * 4) Consume newline if present
00216  *
00217  * @param in Input stream
00218  * @param textOut Output record text (CSV-like)
00219  * @return true if record was read, false if EOF or format error
00220  */
00221 static bool readLenLine(istream& in, string& textOut) {
00222     textOut.clear();
00223
00224     char lenBuf[10];

```

```

00225     if (!in.read(lenBuf, 10)) return false; // EOF or error
00226
00227     for (int i = 0; i < 10; i++) {
00228         if (!isdigit(static_cast<unsigned char>(lenBuf[i]))) return false;
00229     }
00230
00231     char space;
00232     if (!in.get(space) || space != ' ') return false;
00233
00234     int len = stoi(string(lenBuf, 10));
00235     if (len < 0) return false;
00236
00237     textOut.resize(len);
00238     if (len > 0 && !in.read(&textOut[0], len)) return false;
00239
00240     // Consume newline if present
00241     if (in.peek() == '\n') in.get();
00242     else if (in.peek() == '\r') { in.get(); if (in.peek() == '\n') in.get(); }
00243
00244     return true;
00245 }
00246
00247 /* =====
00248 *  SIMPLE CSV FIELD SPLIT (handles basic quotes)
00249 *  =====
00250 */
00251
00252 /**
00253 * @brief Split a CSV line into fields (basic quote support).
00254 * @param line CSV line
00255 * @return vector of fields
00256 */
00257 static vector<string> splitCsvSimple(const string& line) {
00258     vector<string> fields;
00259     string cur;
00260     bool inQuotes = false;
00261
00262     for (char c : line) {
00263         if (c == '"') {
00264             inQuotes = !inQuotes;
00265         } else if (c == ',' && !inQuotes) {
00266             fields.push_back(cur);
00267             cur.clear();
00268         } else {
00269             cur.push_back(c);
00270         }
00271     }
00272     fields.push_back(cur);
00273     return fields;
00274 }
00275
00276 /**
00277 * @brief Print one record with labels on ONE line (Part II requirement).
00278 * @param csvLine The record data (CSV text inside LEN)
00279 *
00280 * Expected order (for now):
00281 * ZipCode,PlaceName,State,County,Lat,Long
00282 *
00283 * If you later support column re-ordering using header metadata,
00284 * you will change this to print by field names from the header mapping.
00285 */
00286 static void printLabeledOneLine(const string& csvLine) {
00287     vector<string> f = splitCsvSimple(csvLine);
00288
00289     for (auto& field : f) {
00290         if (!field.empty() && field.back() == '\r')
00291             field.pop_back();
00292     }
00293
00294     // Remove extra empty field caused by a trailing comma
00295     if (!f.empty() && f.back().empty()) {
00296         f.pop_back();
00297     }
00298
00299     if (f.size() != 6) {
00300         cout << "Record=" << csvLine << "\n";
00301         return;
00302     }
00303
00304     cout << "ZIP=" << f[0]

```

```

00306         « " | Place=" « f[1]
00307         « " | State=" « f[2]
00308         « " | County=" « f[3]
00309         « " | Lat=" « f[4]
00310         « " | Long=" « f[5]
00311         « "\n";
00312     }
00313
00314     /* =====
00315     *  MODE 1: CSV ANALYZE (STREAMING, NO gatherAllRecords)
00316     *  =====
00317     */
00318
00319     /**
00320     * @brief Analyze state extremes from a CSV file without loading all records.
00321     * @param csvFile Input CSV filename
00322     * @return exit code (0 success)
00323     */
00324     static int analyzeCsvStreaming(const string& csvFile) {
00325         ZipCodeBuffer buffer;
00326         if (!buffer.open(csvFile)) {
00327             cerr << "Error: Could not open CSV file '" << csvFile << "'\n";
00328             return 2;
00329         }
00330
00331         map<string, StateExtremes> stateMap;
00332
00333         ZipCodeRecord rec;
00334         long long count = 0;
00335
00336         while (buffer.readRecord(rec)) {
00337             updateStateExtremes(stateMap, rec);
00338             count++;
00339         }
00340
00341         buffer.close();
00342
00343         if (count == 0) {
00344             cerr << "Error: No valid records found.\n";
00345             return 3;
00346         }
00347
00348         cout << "Reading ZIP code data from: " << csvFile << "\n";
00349         cout << "Total records read: " << count << "\n\n";
00350         cout << "Analysis Results:\n===== \n\n";
00351         printStateExtremesTable(stateMap);
00352
00353         return 0;
00354     }
00355
00356     /* =====
00357     *  MODE 2: MAKE LEN FILE FROM CSV
00358     *  =====
00359     */
00360
00361     /**
00362     * @brief Convert CSV → LEN file.
00363     *
00364     * We skip the CSV header row, then write:
00365     * 1) a LEN header record (simple placeholder text)
00366     * 2) each CSV record as a length-indicated record
00367     *
00368     * @param csvFile Input CSV
00369     * @param lenFile Output LEN
00370     * @return exit code
00371     */
00372     static int makeLenFromCsv(const string& csvFile, const string& lenFile) {
00373         ifstream in(csvFile);
00374         if (!in) {
00375             cerr << "Error: Cannot open CSV file '" << csvFile << "'\n";
00376             return 2;
00377         }
00378
00379         ofstream out(lenFile);
00380         if (!out) {
00381             cerr << "Error: Cannot create LEN file '" << lenFile << "'\n";
00382             return 3;
00383         }
00384
00385         // Skip CSV header
00386         string header;

```

```

00387     if (!getline(in, header)) {
00388         cerr << "Error: CSV file is empty.\n";
00389         return 4;
00390     }
00391
00392     // Write full header record using HeaderBuffer class
00393     HeaderBuffer hbuf;
00394     hbuf.buildDefault(lenFile + ".idx", 0);
00395     if (!hbuf.write(out)) {
00396         cerr << "Error: Failed to write LEN header.\n";
00397         return 5;
00398     }
00399
00400     long long recCount = 0;
00401     string line;
00402     while (getline(in, line)) {
00403         if (line.empty()) continue;
00404         if (!writeLenLine(out, line)) {
00405             cerr << "Warning: skipped a line that could not be written.\n";
00406             continue;
00407         }
00408         recCount++;
00409     }
00410
00411     cout << "Created LEN file: " << lenFile << "\n";
00412     cout << "Records written: " << recCount << "\n";
00413     return 0;
00414 }
00415
00416 /* =====
00417  *  MODE 3: BUILD INDEX FROM LEN FILE
00418  *  =====
00419  */
00420
00421 /**
00422  * @brief Build a primary key index (ZIP → byte offset) from a LEN file.
00423  *
00424  * We store the file position BEFORE reading each record.
00425  * That position is the "start of record" (the 10-digit length field).
00426  *
00427  * Index file format (simple):
00428  *   IDX,1
00429  *   56301 128
00430  *   02139 412
00431  *
00432  * @param lenFile Input LEN data file
00433  * @param idxFile Output index file
00434  * @return exit code
00435  */
00436 static int buildIndexFromLen(const string& lenFile, const string& idxFile) {
00437     ifstream in(lenFile);
00438     if (!in) {
00439         cerr << "Error: Cannot open LEN file '" << lenFile << "'\n";
00440         return 2;
00441     }
00442
00443     ofstream out(idxFile);
00444     if (!out) {
00445         cerr << "Error: Cannot create index file '" << idxFile << "'\n";
00446         return 3;
00447     }
00448
00449     string header;
00450     if (!readLenLine(in, header)) {
00451         cerr << "Error: LEN file is missing header or is corrupted.\n";
00452         return 4;
00453     }
00454
00455     out << "IDX,1\n";
00456
00457     long long entries = 0;
00458     while (true) {
00459         streampos pos = in.tellg();
00460
00461         string record;
00462         if (!readLenLine(in, record)) break;
00463
00464         // ZIP is first field before comma
00465         size_t comma = record.find(',');
00466         if (comma == string::npos) continue;
00467     }

```

```

00468         string zip = record.substr(0, comma); // keep leading zeros if any
00469         out << zip << " " << static_cast<long long>(pos) << "\n";
00470         entries++;
00471     }
00472
00473     cout << "Created index file: " << idxFile << "\n";
00474     cout << "Index entries: " << entries << "\n";
00475     return 0;
00476 }
00477
00478 /* =====
00479  * MODE 4: SEARCH USING LEN + INDEX
00480  * =====
00481  */
00482
00483 /**
00484  * @brief Load an index file into RAM.
00485  * @param idxFile Index filename
00486  * @return map ZIP(string) → offset
00487  */
00488 static unordered_map<string, streampos> loadIndex(const string& idxFile) {
00489     unordered_map<string, streampos> idx;
00490
00491     ifstream in(idxFile);
00492     if (!in) return idx;
00493
00494     string firstLine;
00495     getline(in, firstLine); // IDX,1
00496
00497     string zip;
00498     long long pos;
00499     while (in >> zip >> pos) {
00500         idx[zip] = static_cast<streampos>(pos);
00501     }
00502     return idx;
00503 }
00504
00505 /**
00506  * @brief Search all -Z flags provided and print results.
00507  * @param lenFile Data file (.len)
00508  * @param idxFile Index file (.idx)
00509  * @param zips List of ZIP strings to search
00510  * @return exit code
00511  */
00512 static int searchZips(const string& lenFile,
00513                     const string& idxFile,
00514                     const vector<string>& zips) {
00515     unordered_map<string, streampos> idx = loadIndex(idxFile);
00516     if (idx.empty()) {
00517         cerr << "Error: Index file could not be read or is empty: " << idxFile << "\n";
00518         return 2;
00519     }
00520
00521     ifstream data(lenFile);
00522     if (!data) {
00523         cerr << "Error: Cannot open LEN data file: " << lenFile << "\n";
00524         return 3;
00525     }
00526
00527     // Read and keep header in RAM (allowed)
00528     string header;
00529     if (!readLenLine(data, header)) {
00530         cerr << "Error: LEN data file header missing or corrupted.\n";
00531         return 4;
00532     }
00533
00534     cout << "Using data file: " << lenFile << "\n";
00535     cout << "Using index file: " << idxFile << "\n";
00536     cout << "Header: " << header << "\n\n";
00537
00538     for (const string& zip : zips) {
00539         auto it = idx.find(zip);
00540         if (it == idx.end()) {
00541             cout << "ZIP " << zip << " not found in file\n";
00542             continue;
00543         }
00544
00545         data.clear();
00546         data.seekg(it->second);
00547
00548         string recordLine;

```



```

00549         if (!readLenLine(data, recordLine)) {
00550             cout << "ZIP " << zip << " found in index but record could not be read (stale index)\n";
00551             continue;
00552         }
00553
00554         printLabeledOneLine(recordLine);
00555     }
00556
00557     return 0;
00558 }
00559
00560 /* =====
00561  *  USAGE MESSAGE
00562  *  =====
00563  */
00564
00565 /**
00566  * @brief Print usage instructions (simple).
00567  * @param prog program name
00568  */
00569 static void printUsage(const string& prog) {
00570     cerr << "USAGE:\n";
00571     cerr << "  1) Analyze CSV (Project 1 style):\n";
00572     cerr << "       " << prog << " <file.csv>\n\n";
00573     cerr << "  2) Make LEN from CSV:\n";
00574     cerr << "       " << prog << " --make-len <in.csv> <out.len>\n\n";
00575     cerr << "  3) Build index from LEN:\n";
00576     cerr << "       " << prog << " --build-index <data.len> <out.idx>\n\n";
00577     cerr << "  4) Search ZIPs using LEN + IDX:\n";
00578     cerr << "       " << prog << " --search <data.len> <data.idx> -Z56301 -Z99546 -Z99999\n";
00579 }
00580
00581 /* =====
00582  *  MAIN
00583  *  =====
00584  */
00585
00586 int main(int argc, char* argv[]) {
00587     if (argc < 2) {
00588         printUsage(argv[0]);
00589         return 1;
00590     }
00591
00592     string cmd = argv[1];
00593
00594     // MODE: --make-len in.csv out.len
00595     if (cmd == "--make-len") {
00596         if (argc != 4) {
00597             printUsage(argv[0]);
00598             return 1;
00599         }
00600         return makeLenFromCsv(argv[2], argv[3]);
00601     }
00602
00603     // MODE: --build-index data.len out.idx
00604     if (cmd == "--build-index") {
00605         if (argc != 4) {
00606             printUsage(argv[0]);
00607             return 1;
00608         }
00609         return buildIndexFromLen(argv[2], argv[3]);
00610     }
00611
00612     // MODE: --search data.len data.idx -Zxxxxx ...
00613     if (cmd == "--search") {
00614         if (argc < 5) {
00615             printUsage(argv[0]);
00616             return 1;
00617         }
00618
00619         string lenFile = argv[2];
00620         string idxFile = argv[3];
00621
00622         vector<string> zips;
00623         for (int i = 4; i < argc; i++) {
00624             string arg = argv[i];
00625             if (arg.rfind("-Z", 0) == 0 && arg.size() > 2) {
00626                 zips.push_back(arg.substr(2)); // everything after -Z
00627             }
00628         }
00629     }

```

```

00630         if (zips.empty()) {
00631             cerr << "Error: No -Z flags provided.\n";
00632             printUsage(argv[0]);
00633             return 1;
00634         }
00635
00636         return searchZips(lenFile, idxFile, zips);
00637     }
00638
00639     // DEFAULT MODE: treat argv[1] as CSV file and analyze
00640     // Example: ./zipprog us_postal_codes.csv
00641     return analyzeCsvStreaming(argv[1]);
00642 }

```

4.19 ZipCodeBuffer.cpp File Reference

Implementation of the [ZipCodeBuffer](#) class.

```

#include "ZipCodeBuffer.h"
#include <algorithm>
#include <cctype>
#include <stdexcept>

```

4.19.1 Detailed Description

Implementation of the [ZipCodeBuffer](#) class.

This version supports CSV files even when the columns are re-ordered. It reads the header row first, remembers the needed column positions, and then parses data rows using those positions.

Definition in file [ZipCodeBuffer.cpp](#).

4.20 ZipCodeBuffer.cpp

[Go to the documentation of this file.](#)

```

00001 /**
00002  * @file ZipCodeBuffer.cpp
00003  * @brief Implementation of the ZipCodeBuffer class
00004  *
00005  * This version supports CSV files even when the columns are re-ordered.
00006  * It reads the header row first, remembers the needed column positions,
00007  * and then parses data rows using those positions.
00008  */
00009
00010 #include "ZipCodeBuffer.h"
00011 #include <algorithm>
00012 #include <cctype>
00013 #include <stdexcept>
00014
00015 using namespace std;
00016
00017 /* =====
00018  * ZipCodeRecord Implementation
00019  * =====
00020  */
00021
00022 /**
00023  * @brief Default constructor
00024  */
00025 ZipCodeRecord::ZipCodeRecord()
00026     : zipCode(0), placeName(""), state(""), county(""),
00027       latitude(0.0), longitude(0.0) {}
00028 }
00029
00030 /**
00031  * @brief Parameterized constructor
00032  */
00033 ZipCodeRecord::ZipCodeRecord(int zip, const string& place, const string& st,
00034                               const string& cnty, double lat, double lon)
00035     : zipCode(zip), placeName(place), state(st), county(cnty),
00036       latitude(lat), longitude(lon) {}
00037 }
00038

```

```

00039 /* =====
00040 * ZipCodeBuffer Implementation
00041 * =====
00042 */
00043
00044 /**
00045 * @brief Default constructor
00046 *
00047 * Initializes all member variables to safe starting values.
00048 */
00049 ZipCodeBuffer::ZipCodeBuffer()
00050     : filename(""), headerSkipped(false), recordCount(0),
00051       colZip(-1), colPlace(-1), colState(-1),
00052       colCounty(-1), colLat(-1), colLong(-1) {
00053 }
00054
00055 /**
00056 * @brief Constructor that opens a file immediately
00057 */
00058 ZipCodeBuffer::ZipCodeBuffer(const string& csvFilename)
00059     : filename(""), headerSkipped(false), recordCount(0),
00060       colZip(-1), colPlace(-1), colState(-1),
00061       colCounty(-1), colLat(-1), colLong(-1) {
00062     open(csvFilename);
00063 }
00064
00065 /**
00066 * @brief Destructor
00067 */
00068 ZipCodeBuffer::~ZipCodeBuffer() {
00069     close();
00070 }
00071
00072 /**
00073 * @brief Opens a CSV file and processes its header row
00074 *
00075 * This function:
00076 * 1) closes any previously open file
00077 * 2) opens the requested file
00078 * 3) reads the first line (header row)
00079 * 4) finds the positions of ZIP, place, state, county, latitude, longitude
00080 *
00081 * @param csvFilename CSV file path
00082 * @return true on success, false on failure
00083 */
00084 bool ZipCodeBuffer::open(const string& csvFilename) {
00085     close();
00086
00087     filename = csvFilename;
00088     fileStream.open(filename);
00089
00090     if (!fileStream.is_open()) {
00091         return false;
00092     }
00093
00094     string headerLine;
00095     if (!getline(fileStream, headerLine)) {
00096         close();
00097         return false;
00098     }
00099
00100     if (!parseHeader(headerLine)) {
00101         close();
00102         return false;
00103     }
00104
00105     headerSkipped = true;
00106     recordCount = 0;
00107     return true;
00108 }
00109
00110 /**
00111 * @brief Closes the file and resets internal state
00112 */
00113 void ZipCodeBuffer::close() {
00114     if (fileStream.is_open()) {
00115         fileStream.close();
00116     }
00117
00118     filename = "";
00119     headerSkipped = false;

```

```

00120     recordCount = 0;
00121
00122     colZip = -1;
00123     colPlace = -1;
00124     colState = -1;
00125     colCounty = -1;
00126     colLat = -1;
00127     colLong = -1;
00128 }
00129
00130 /**
00131  * @brief Checks whether file is open
00132  */
00133 bool ZipCodeBuffer::isOpen() const {
00134     return fileStream.is_open();
00135 }
00136
00137 /**
00138  * @brief Reads one data record from the CSV file
00139  *
00140  * @param record Output record
00141  * @return true if record is read successfully, false on EOF or parse failure
00142  */
00143 bool ZipCodeBuffer::readRecord(ZipCodeRecord& record) {
00144     if (!fileStream.is_open()) {
00145         return false;
00146     }
00147
00148     string line;
00149     if (getline(fileStream, line)) {
00150         if (line.empty() || trim(line).empty()) {
00151             return readRecord(record); // skip blank lines
00152         }
00153
00154         if (parseLine(line, record)) {
00155             recordCount++;
00156             return true;
00157         } else {
00158             return false;
00159         }
00160     }
00161
00162     return false;
00163 }
00164
00165 /**
00166  * @brief Reads all records into a vector
00167  *
00168  * This is kept for compatibility, but Project 2 should prefer reading
00169  * one record at a time.
00170  */
00171 vector<ZipCodeRecord> ZipCodeBuffer::gatherAllRecords() {
00172     vector<ZipCodeRecord> records;
00173
00174     reset();
00175
00176     ZipCodeRecord record;
00177     while (readRecord(record)) {
00178         records.push_back(record);
00179     }
00180
00181     reset();
00182     return records;
00183 }
00184
00185 /**
00186  * @brief Resets file back to beginning and processes header row again
00187  *
00188  * @return true if reset succeeds
00189  */
00190 bool ZipCodeBuffer::reset() {
00191     if (!fileStream.is_open()) {
00192         return false;
00193     }
00194
00195     fileStream.clear();
00196     fileStream.seekg(0, ios::beg);
00197
00198     string headerLine;
00199     if (!getline(fileStream, headerLine)) {
00200         return false;

```

```

00201     }
00202
00203     if (!parseHeader(headerLine)) {
00204         return false;
00205     }
00206
00207     recordCount = 0;
00208     headerSkipped = true;
00209     return true;
00210 }
00211
00212 /**
00213  * @brief Returns how many records have been read
00214  */
00215 long ZipCodeBuffer::getRecordCount() const {
00216     return recordCount;
00217 }
00218
00219 /**
00220  * @brief Returns current filename
00221  */
00222 string ZipCodeBuffer::getFilename() const {
00223     return filename;
00224 }
00225
00226 /**
00227  * @brief Parses the CSV header row and finds column positions
00228  *
00229  * This function makes column re-ordering possible.
00230  *
00231  * For example, if the header is:
00232  * PlaceName,ZipCode,County,State,Long,Lat
00233  *
00234  * then this function will correctly discover:
00235  * - colPlace = 0
00236  * - colZip = 1
00237  * - colCounty = 2
00238  * - colState = 3
00239  * - colLong = 4
00240  * - colLat = 5
00241  *
00242  * @param headerLine Header row from the CSV file
00243  * @return true if all needed columns are found
00244  */
00245 bool ZipCodeBuffer::parseHeader(const string& headerLine) {
00246     vector<string> fields = splitCSV(headerLine);
00247
00248     colZip = -1;
00249     colPlace = -1;
00250     colState = -1;
00251     colCounty = -1;
00252     colLat = -1;
00253     colLong = -1;
00254
00255     for (size_t i = 0; i < fields.size(); i++) {
00256         string name = trim(fields[i]);
00257
00258         // Make comparison easier by converting to lowercase
00259         string lowerName = name;
00260         transform(lowerName.begin(), lowerName.end(), lowerName.begin(),
00261             [](unsigned char c) { return static_cast<char>(tolower(c)); });
00262
00263         if (lowerName == "zipcode" || lowerName == "zip" || lowerName == "zip_code") {
00264             colZip = static_cast<int>(i);
00265         } else if (lowerName == "placename" || lowerName == "place" || lowerName == "city") {
00266             colPlace = static_cast<int>(i);
00267         } else if (lowerName == "state") {
00268             colState = static_cast<int>(i);
00269         } else if (lowerName == "county") {
00270             colCounty = static_cast<int>(i);
00271         } else if (lowerName == "lat" || lowerName == "latitude") {
00272             colLat = static_cast<int>(i);
00273         } else if (lowerName == "long" || lowerName == "longitude" || lowerName == "lon") {
00274             colLong = static_cast<int>(i);
00275         }
00276     }
00277
00278     return (colZip != -1 &&
00279         colPlace != -1 &&
00280         colState != -1 &&
00281         colCounty != -1 &&

```

```

00282         collat != -1 &&
00283         collong != -1);
00284     }
00285
00286 /**
00287  * @brief Parses one CSV data line into a ZipCodeRecord
00288  *
00289  * This version does NOT assume fixed column order.
00290  * Instead, it uses the column positions discovered from the header row.
00291  *
00292  * @param line One CSV data row
00293  * @param record Output record
00294  * @return true if parsing succeeds
00295  */
00296 bool ZipCodeBuffer::parseLine(const string& line, ZipCodeRecord& record) {
00297     vector<string> fields = splitCSV(line);
00298
00299     int maxNeeded = max({colZip, colPlace, colState, colCounty, collat, collong});
00300     if (static_cast<int>(fields.size()) <= maxNeeded) {
00301         return false;
00302     }
00303
00304     try {
00305         record.zipCode = stoi(trim(fields[colZip]));
00306         record.placeName = trim(fields[colPlace]);
00307         record.state = trim(fields[colState]);
00308         record.county = trim(fields[colCounty]);
00309         record.latitude = stod(trim(fields[collat]));
00310         record.longitude = stod(trim(fields[collong]));
00311
00312         return true;
00313     } catch (const exception&) {
00314         return false;
00315     }
00316 }
00317
00318 /**
00319  * @brief Splits a CSV line into fields
00320  *
00321  * Handles:
00322  * - normal commas
00323  * - quoted text
00324  * - commas inside quoted text
00325  *
00326  * @param line One CSV line
00327  * @return Vector of fields
00328  */
00329 vector<string> ZipCodeBuffer::splitCSV(const string& line) {
00330     vector<string> fields;
00331     string currentField;
00332     bool inQuotes = false;
00333
00334     for (size_t i = 0; i < line.length(); i++) {
00335         char c = line[i];
00336
00337         if (c == '"') {
00338             inQuotes = !inQuotes;
00339         } else if (c == ',' && !inQuotes) {
00340             fields.push_back(currentField);
00341             currentField.clear();
00342         } else {
00343             currentField += c;
00344         }
00345     }
00346
00347     fields.push_back(currentField);
00348     return fields;
00349 }
00350
00351 /**
00352  * @brief Trims whitespace from both ends of a string
00353  *
00354  * @param str Input string
00355  * @return Trimmed string
00356  */
00357 string ZipCodeBuffer::trim(const string& str) {
00358     size_t start = str.find_first_not_of(" \t\r\n");
00359     if (start == string::npos) {
00360         return "";
00361     }
00362

```

```
00363     size_t end = str.find_last_not_of(" \t\r\n");
00364     return str.substr(start, end - start + 1);
00365 }
```

4.21 ZipCodeBuffer.h File Reference

Header file for the [ZipCodeBuffer](#) class and [ZipCodeRecord](#) structure.

```
#include <string>
#include <fstream>
#include <sstream>
#include <vector>
```

Classes

- struct [ZipCodeRecord](#)
Structure to hold a single ZIP code record.
- class [ZipCodeBuffer](#)
A buffer class for reading ZIP code records from CSV files.

4.21.1 Detailed Description

Header file for the [ZipCodeBuffer](#) class and [ZipCodeRecord](#) structure.

Author

Teagen Lee (primary contributor)
Ethan Jackson (additional comments and formatting)
Dristi Barnwal, Marcus Julius, Natoli Mayu (reviewers)

Date

February 2026

This file contains:

- [ZipCodeRecord](#): a structure that stores one ZIP code record
- [ZipCodeBuffer](#): a class that reads ZIP code data from a CSV file

Project 2 improvement: This version supports CSV files whose columns may appear in different orders. It does this by reading the header row first and finding which column contains:

- ZIP code
- place name
- state
- county
- latitude
- longitude

Definition in file [ZipCodeBuffer.h](#).

4.22 ZipCodeBuffer.h

[Go to the documentation of this file.](#)

```

00001 /**
00002  * @file ZipCodeBuffer.h
00003  * @brief Header file for the ZipCodeBuffer class and ZipCodeRecord structure
00004  * @author Teagen Lee (primary contributor)
00005  * @author Ethan Jackson (additional comments and formatting)
00006  * @author Dristi Barnwal, Marcus Julius, Natoli Mayu (reviewers)
00007  * @date February 2026
00008  *
00009  * This file contains:
00010  * - ZipCodeRecord: a structure that stores one ZIP code record
00011  * - ZipCodeBuffer: a class that reads ZIP code data from a CSV file
00012  *
00013  * Project 2 improvement:
00014  * This version supports CSV files whose columns may appear in different orders.
00015  * It does this by reading the header row first and finding which column contains:
00016  * - ZIP code
00017  * - place name
00018  * - state
00019  * - county
00020  * - latitude
00021  * - longitude
00022  */
00023
00024 #ifndef ZIPCODEBUFFER_H
00025 #define ZIPCODEBUFFER_H
00026
00027 #include <string>
00028 #include <fstream>
00029 #include <sstream>
00030 #include <vector>
00031
00032 using namespace std;
00033
00034 /**
00035  * @struct ZipCodeRecord
00036  * @brief Structure to hold a single ZIP code record
00037  *
00038  * This structure stores one full data row from the ZIP code file.
00039  */
00040 struct ZipCodeRecord {
00041     int zipCode;           ///< The 5-digit ZIP code
00042     string placeName;      ///< Name of the place/city
00043     string state;          ///< Two-letter state abbreviation
00044     string county;         ///< County name
00045     double latitude;       ///< Latitude coordinate
00046     double longitude;      ///< Longitude coordinate
00047
00048     /**
00049      * @brief Default constructor
00050      *
00051      * Initializes all values to safe defaults.
00052      */
00053     ZipCodeRecord();
00054
00055     /**
00056      * @brief Parameterized constructor
00057      * @param zip ZIP code number
00058      * @param place Place name
00059      * @param st State abbreviation
00060      * @param cnty County name
00061      * @param lat Latitude
00062      * @param lon Longitude
00063      */
00064     ZipCodeRecord(int zip, const string& place, const string& st,
00065                   const string& cnty, double lat, double lon);
00066 };
00067
00068 /**
00069  * @class ZipCodeBuffer
00070  * @brief A buffer class for reading ZIP code records from CSV files
00071  *
00072  * This class opens a CSV file, reads one record at a time, and converts each
00073  * line into a ZipCodeRecord object.
00074  *
00075  * Project 2 improvement:
00076  * The class does not assume that columns are always in the same order.
00077  * Instead, it reads the header row and finds where each needed field is.

```



```

00078  */
00079  class ZipCodeBuffer {
00080  private:
00081      ifstream fileStream;          ///< Input file stream for CSV data
00082      string filename;              ///< Name of the current file
00083      bool headerSkipped;           ///< True after header row is processed
00084      long recordCount;             ///< Number of records successfully read
00085
00086      /**
00087       * Column positions found from the CSV header row.
00088       *
00089       * These values tell the parser where each needed field is located.
00090       * Example:
00091       * - colZip = 0 means ZIP code is in column 0
00092       * - colState = 2 means state is in column 2
00093       *
00094       * If the columns are re-ordered, these values will change, and parsing
00095       * will still work correctly.
00096       */
00097      int colZip;
00098      int colPlace;
00099      int colState;
00100      int colCounty;
00101      int colLat;
00102      int colLong;
00103
00104      /**
00105       * @brief Parses a CSV line into a ZipCodeRecord
00106       * @param line The CSV line to parse
00107       * @param record Reference to record to fill
00108       * @return true if parsing succeeds, false otherwise
00109       */
00110      bool parseLine(const string& line, ZipCodeRecord& record);
00111
00112      /**
00113       * @brief Splits a CSV line into separate fields
00114       * @param line One CSV line
00115       * @return Vector of field strings
00116       */
00117      vector<string> splitCSV(const string& line);
00118
00119      /**
00120       * @brief Removes whitespace from both ends of a string
00121       * @param str Input string
00122       * @return Trimmed string
00123       */
00124      string trim(const string& str);
00125
00126      /**
00127       * @brief Reads the header row and finds the column position of each field
00128       * @param headerLine The first line of the CSV file
00129       * @return true if all required columns were found, false otherwise
00130       *
00131       * This function makes the class work even when the CSV columns are
00132       * re-ordered.
00133       */
00134      bool parseHeader(const string& headerLine);
00135
00136  public:
00137      /**
00138       * @brief Default constructor
00139       */
00140      ZipCodeBuffer();
00141
00142      /**
00143       * @brief Constructor that immediately opens a CSV file
00144       * @param csvFilename Path to CSV file
00145       */
00146      explicit ZipCodeBuffer(const string& csvFilename);
00147
00148      /**
00149       * @brief Destructor
00150       */
00151      ~ZipCodeBuffer();
00152
00153      /**
00154       * @brief Opens a CSV file for reading
00155       * @param csvFilename Path to CSV file
00156       * @return true if file opened successfully, false otherwise
00157       */
00158      bool open(const string& csvFilename);

```

```
00159
00160 /**
00161  * @brief Closes the file and resets internal state
00162  */
00163 void close();
00164
00165 /**
00166  * @brief Checks whether a file is currently open
00167  * @return true if file is open
00168  */
00169 bool isOpen() const;
00170
00171 /**
00172  * @brief Reads the next ZIP code record from the file
00173  * @param record Output record
00174  * @return true if record was read successfully, false on EOF or error
00175  */
00176 bool readRecord(ZipCodeRecord& record);
00177
00178 /**
00179  * @brief Reads all records into a vector
00180  * @return Vector of all records
00181  *
00182  * Note:
00183  * This is kept mainly for compatibility with older code, but Project 2
00184  * should prefer reading one record at a time.
00185  */
00186 vector<ZipCodeRecord> gatherAllRecords();
00187
00188 /**
00189  * @brief Resets the file position to the beginning, then processes header
00190  * @return true if reset succeeds, false otherwise
00191  */
00192 bool reset();
00193
00194 /**
00195  * @brief Gets the number of records read so far
00196  * @return Record count
00197  */
00198 long getRecordCount() const;
00199
00200 /**
00201  * @brief Gets the current filename
00202  * @return Filename
00203  */
00204 string getFilename() const;
00205 };
00206
00207 #endif
```

Index

- ~ZipCodeBuffer
 - ZipCodeBuffer, [17](#)
- analyzeCsvStreaming
 - main.cpp, [51](#)
- buildDefault
 - HeaderBuffer, [7](#)
- buildIndex
 - IndexBuilder.cpp, [35](#)
 - IndexBuilder.h, [38](#)
- buildIndexFromLen
 - main.cpp, [51](#)
- close
 - ZipCodeBuffer, [17](#)
- colCounty
 - ZipCodeBuffer, [24](#)
- colLat
 - ZipCodeBuffer, [24](#)
- colLong
 - ZipCodeBuffer, [24](#)
- colPlace
 - ZipCodeBuffer, [24](#)
- colState
 - ZipCodeBuffer, [25](#)
- colZip
 - ZipCodeBuffer, [25](#)
- county
 - ZipCodeRecord, [28](#)
- deserialize
 - HeaderBuffer, [8](#)
- easternmost
 - StateExtremes, [13](#)
- fieldCount
 - FileHeader, [4](#)
- fieldNames
 - FileHeader, [4](#)
- fieldTypes
 - FileHeader, [4](#)
- FileHeader, [3](#)
 - fieldCount, [4](#)
 - fieldNames, [4](#)
 - fieldTypes, [4](#)
 - fileType, [4](#)
 - headerSizeBytes, [4](#)
 - indexFileName, [4](#)
 - primaryKeyFieldIndex, [5](#)
 - recordCount, [5](#)
 - recordSizeByteCount, [5](#)
 - sizeFormatType, [5](#)
 - sizeIncludesItself, [5](#)
 - sizeOfSizes, [5](#)
 - staleIndex, [6](#)
 - version, [6](#)
- filename
 - ZipCodeBuffer, [25](#)
- fileStream
 - ZipCodeBuffer, [25](#)
- fileType
 - FileHeader, [4](#)
- gatherAllRecords
 - ZipCodeBuffer, [17](#)
- getFilename
 - ZipCodeBuffer, [18](#)
- getHeader
 - HeaderBuffer, [8](#)
- getRecordCount
 - ZipCodeBuffer, [18](#)
- header_
 - HeaderBuffer, [12](#)
- HeaderBuffer, [6](#)
 - buildDefault, [7](#)
 - deserialize, [8](#)
 - getHeader, [8](#)
 - header_, [12](#)
 - HeaderBuffer, [7](#)
 - print, [8](#)
 - read, [9](#)
 - readLenLine, [9](#)
 - serialize, [10](#)
 - write, [10](#)
 - writeLenLine, [10](#)
- HeaderBuffer.cpp, [29](#)
- HeaderBuffer.h, [33](#)
- headerSizeBytes
 - FileHeader, [4](#)
- headerSkipped
 - ZipCodeBuffer, [26](#)
- IndexBuilder.cpp, [34](#)
 - buildIndex, [35](#)
- IndexBuilder.h, [37](#)
 - buildIndex, [38](#)
- indexFileName
 - FileHeader, [4](#)
- isOpen
 - ZipCodeBuffer, [18](#)

- latitude
 - ZipCodeRecord, 28
- LenFileReader.cpp, 39
 - readLenRecord, 39
- LenFileReader.h, 41
 - readLenRecord, 42
- LenFileWriter.cpp, 43
 - makeLenFile, 44
 - writeLenLine, 44
- LenFileWriter.h, 47
 - makeLenFile, 48
- loadIndex
 - main.cpp, 52
- longitude
 - ZipCodeRecord, 28
- main
 - main.cpp, 52
- main.cpp, 49
 - analyzeCsvStreaming, 51
 - buildIndexFromLen, 51
 - loadIndex, 52
 - main, 52
 - makeLenFromCsv, 52
 - printLabeledOneLine, 53
 - printStateExtremesTable, 53
 - printUsage, 53
 - readLenLine, 53
 - searchZips, 54
 - smallerZipWins, 54
 - splitCsvSimple, 55
 - updateStateExtremes, 55
 - writeLenLine, 55
- makeLenFile
 - LenFileWriter.cpp, 44
 - LenFileWriter.h, 48
- makeLenFromCsv
 - main.cpp, 52
- maxLatitude
 - StateExtremes, 13
- maxLongitude
 - StateExtremes, 13
- minLatitude
 - StateExtremes, 14
- minLongitude
 - StateExtremes, 14
- northernmost
 - StateExtremes, 14
- open
 - ZipCodeBuffer, 19
- parseHeader
 - ZipCodeBuffer, 19
- parseLine
 - ZipCodeBuffer, 20
- placeName
 - ZipCodeRecord, 28
- primaryKeyFieldIndex
 - FileHeader, 5
- print
 - HeaderBuffer, 8
- printLabeledOneLine
 - main.cpp, 53
- printStateExtremesTable
 - main.cpp, 53
- printUsage
 - main.cpp, 53
- read
 - HeaderBuffer, 9
- readLenLine
 - HeaderBuffer, 9
 - main.cpp, 53
- readLenRecord
 - LenFileReader.cpp, 39
 - LenFileReader.h, 42
- readRecord
 - ZipCodeBuffer, 22
- recordCount
 - FileHeader, 5
 - ZipCodeBuffer, 26
- recordSizeByteCount
 - FileHeader, 5
- reset
 - ZipCodeBuffer, 22
- searchZips
 - main.cpp, 54
- serialize
 - HeaderBuffer, 10
- sizeFormatType
 - FileHeader, 5
- sizeIncludesItself
 - FileHeader, 5
- sizeOfSizes
 - FileHeader, 5
- smallerZipWins
 - main.cpp, 54
- southernmost
 - StateExtremes, 14
- splitCSV
 - ZipCodeBuffer, 23
- splitCsvSimple
 - main.cpp, 55
- staleIndex
 - FileHeader, 6
- state
 - ZipCodeRecord, 29

- StateExtremes, [12](#)
 - easternmost, [13](#)
 - maxLatitude, [13](#)
 - maxLongitude, [13](#)
 - minLatitude, [14](#)
 - minLongitude, [14](#)
 - northernmost, [14](#)
 - southernmost, [14](#)
 - StateExtremes, [13](#)
 - westernmost, [14](#)
- trim
 - ZipCodeBuffer, [23](#)
- updateStateExtremes
 - main.cpp, [55](#)
- version
 - FileHeader, [6](#)
- westernmost
 - StateExtremes, [14](#)
- write
 - HeaderBuffer, [10](#)
- writeLenLine
 - HeaderBuffer, [10](#)
 - LenFileWriter.cpp, [44](#)
 - main.cpp, [55](#)
- zipCode
 - ZipCodeRecord, [29](#)
- ZipCodeBuffer, [15](#)
 - ~ZipCodeBuffer, [17](#)
 - close, [17](#)
 - colCounty, [24](#)
 - colLat, [24](#)
 - colLong, [24](#)
 - colPlace, [24](#)
 - colState, [25](#)
 - colZip, [25](#)
 - filename, [25](#)
 - fileStream, [25](#)
 - gatherAllRecords, [17](#)
 - getFilename, [18](#)
 - getRecordCount, [18](#)
 - headerSkipped, [26](#)
 - isOpen, [18](#)
 - open, [19](#)
 - parseHeader, [19](#)
 - parseLine, [20](#)
 - readRecord, [22](#)
 - recordCount, [26](#)
 - reset, [22](#)
 - splitCSV, [23](#)
 - trim, [23](#)
 - ZipCodeBuffer, [16](#)
 - ZipCodeBuffer.cpp, [64](#)
 - ZipCodeBuffer.h, [69](#)
 - ZipCodeRecord, [26](#)
 - county, [28](#)
 - latitude, [28](#)
 - longitude, [28](#)
 - placeName, [28](#)
 - state, [29](#)
 - zipCode, [29](#)
 - ZipCodeRecord, [27](#)